

Complementos de Bases de Dados 2025/2026

Licenciatura em Eng^a. Informática

1^a Fase Relatório Técnico

Turma: PL3

Horário de Laboratório: Quarta – Feira 16:30/18:30

Docente: Cláudio Sapateiro

Grupo

Nº202300160, Guilherme do Nascimento Garcia

Nº202200315, Samuel Silva

1. Introdução	4
2. Especificação de Requisitos	4
3. Modelo Relacional (Modelo de dados)	5
3.1.Diagrama do Modelo Entidade Relação	13
3.2.Diagrama do Modelo Relacional	14
4. Definição do Layout	15
4.1.Identificação do espaço ocupado por tabela	16
4.2.Especificação dos Filegroups	18
4.3.Schemas	19
5. Verificação da migração de dados	19
5.1.Consultas sobre a base de dados original	20
5.2.Consultas sobre a nova base de dados	31
6. Programação	37
6.1.Views	37
6.2.Functions	38
6.3.Stored procedures	39
6.4.Triggers	40
7. Catálogo/Metadados	41
7.1.Monitorização	41
8. Índices	42
9. Backup e Recuperação	50
1. Modelo de Recuperação	50
2. Tipo de backup	50
Tipos de backup a implementar:	51
Dispositivos de armazenamento	52
Simulação de crash e recuperação de base de dados	52
Simulação de “Crash”	54

SEQUÊNCIA DE RECUPERAÇÃO	62
10. de Acesso à Informação	Níveis 66
1. Administrador	66
2. SalesPerson	68
3. SalesTerritory	69
11. tação	Encrip 70
Campos Sensíveis	70
Consulta de NIF Encriptado e Desencriptado:	72
Consulta password hash	75
12. lo de Transações	Contro 76
Cenário de Conflito – Consulta e Reserva de Stock	76
Estratégia de Controlo de Concorrência	78
Demonstração	79
Demonstração do Conflito (Sem Controlo)	79
Solução com Bloqueio Pessimista	81
13. L	NoSQ 82
14. ção da Demonstração	Descri 88
14.1.Script de demonstração	89
15. sões	Conclu 90

1. Introdução

O presente projeto tem como principal objetivo o desenvolvimento e migração de uma base de dados relacional. O trabalho incide sobre o processo de transição dos dados de um sistema legado (base de dados AdventureWorksLegacy) para uma nova estrutura normalizada e otimizada (AdventureWorks), assegurando a integridade, consistência e qualidade dos dados migrados.

O projeto engloba várias fases interdependentes, desde a análise do modelo de dados original, o planeamento da estrutura destino, a definição de chaves primárias e estrangeiras, até à implementação de scripts de migração desenvolvidos em Transact-SQL (T-SQL). Foram também implementadas funções, triggers e stored procedures com o propósito de apoiar o processo de migração e a validação dos dados carregados.

Este projeto visa assim simular um processo real de migração de dados empresariais, combinando práticas de modelação, programação SQL e gestão da integridade referencial, com ênfase na segurança e fiabilidade do sistema resultante.

2. Especificação de Requisitos

Especificar os requisitos funcionais apresentados no enunciado do projeto. Estes requisitos devem incluir restrições de integridade ou regras de validação de informação/processos de negócio. Os requisitos que são propostos como melhoria aos expostos no enunciado devem ser incluídos e identificados por RM##.

ID	Descrição	Implementação (S/N)
R01	O sistema deverá permitir a migração completa dos dados do sistema legado (AdventureWorksLegacy) para a nova base de dados (AdventureWorks), sem perda de informação.	S
R02	O sistema deverá garantir a integridade referencial entre todas as tabelas migradas, assegurando que não existam registos órfãos (por exemplo, SalesOrder sem Customer associado).	S
R03	Deverão ser implementadas stored procedures e functions para apoiar a migração e verificação da qualidade dos dados.	S
R04	O sistema deverá permitir a gestão de utilizadores, incluindo a criação, edição e remoção de contas (AppUser).	S
R05	Deverá ser implementada a funcionalidade de recuperação de password, incluindo questão e resposta de segurança, com registo na tabela PasswordRecoveryQuestion.	S
R06	As passwords e NIF dos clientes devem ser armazenados de forma encriptada ou com hash, garantindo a confidencialidade da informação sensível.	N

1ª Fase Relatório Técnico – Complementos de Bases de Dados

R07	Deverá ser criada a tabela SentEmails, simulando o envio automático de mensagens de recuperação de password, com registo de destinatário, mensagem e data/hora.	S
R08	O sistema deverá disponibilizar consultas de validação para confirmar a correspondência de dados entre a base de dados original e a nova.	S
R09	O sistema deverá incluir uma stored procedure sp_dbstatistics que registe, para cada tabela, o número de registos, o espaço ocupado e a data da última atualização, mantendo histórico.	S
R10	Deverão ser criadas functions utilitárias para apoio à verificação e qualidade dos dados migrados.	S
R11	O género de um Customer é opcional e pode não ser inserido	S

3. Modelo Relacional (*Modelo de dados*)

ProductMaster

Representa o produto base (ex.: “HL Touring Frame”)

- `product\_master\_id` (PK)
- `product\_name` (ex.: “HL Touring Frame”)
- `model` (ex.: “HL Touring Frame”)
- `category\_id` (FK → ProductCategory)
- `subcategory\_id` (FK → ProductSubcategory)
- `product\_line\_id` (FK → ProductLine, opcional)
- `class\_id` (FK → ProductClass, opcional)
- `description` (ex.: “The HL aluminum frame is custom-shaped for strength and durability.”)

ProductVariant

Representa uma variação específica de um produto (cor, tamanho, peso, preço).

- `product\_variant\_id` (PK)
- `product\_master\_id` (FK → ProductMaster)
- `variant\_name` (ex.: “HL Touring Frame – Red”)
- `color\_id` (FK → ProductColor, opcional)
- `style\_id` (FK → ProductStyle, opcional)
- `size` (ex.: 'S', 'L', '60')

1ª Fase Relatório Técnico – Complementos de Bases de Dados

- `size\_range\_id` (FK → ProductSizeRange, opcional)
- `size\_unit\_measure\_code` (FK → UnitOfMeasure, opcional)
- `weight` (DECIMAL, ex.: 3.08, opcional)
- `weight\_unit\_measure\_code` (FK → UnitOfMeasure, opcional)
- `finished\_goods\_flag` (BIT)
- `standard\_cost` (DECIMAL(10,2))
- `list\_price` (DECIMAL(10,2))
- `dealer\_price` (DECIMAL(10,2))
- `days\_to\_manufacture` (INT)
- `safety\_stock\_level` (INT)

ProductColor

Representa a cor de um produto.

- `color\_id` (PK)
- `name` (ex.: “Blue”, “Yellow”)

ProductCategory

Representa a categoria do produto.

- `category\_id` (PK)
- `name` (ex.: “Components”)

ProductSubcategory

Representa a subcategoria de um produto.

- `subcategory\_id` (PK)
- `name` (ex.: “Frames”)

ProductLine

Representa a linha do produto.

- `product\_line\_id` (PK)
- `name` (ex.: “T”)

ProductClass

Representa a classe do produto.

- `class\_id` (PK)
- `name` (ex.: “H”)

Ano Letivo 2025/26

ProductStyle

Representa o estilo do produto.

- `style\_id` (PK)
- `name` (ex.: “U”)

ProductSizeRange

Representa a gama de tamanhos de um produto.

- `size\_range\_id` (PK)
- `name` (ex.: “60-62 CM”)

UnitOfMeasure

Representa as unidades de medida.

- `unit\_measure\_code` (PK)
- `name` (ex.: “Pounds”, “Centimeters”)
- `conversion\_to\_base` (DECIMAL(10,6), ex.: 0.453592 para LB → KG)

Customer

Representa um cliente.

- `customer\_id` (PK)
- `title` (ex.: “Mr.”)
- `first\_name` (ex.: “Jon”)
- `middle\_name` (ex.: “V”)
- `last\_name` (ex.: “Yang”)
- `birth\_date` (DATE)
- `marital\_status` (CHAR(1))
- `gender` (CHAR(1), opcional)
- `email\_address` (NVARCHAR(100))
- `yearly\_income` (DECIMAL(10,2))
- `education` (NVARCHAR(50))
- `occupation` (NVARCHAR(50))
- `number\_cars\_owned` (INT)
- `date\_first\_purchase` (DATE)
- `nif` (NVARCHAR(20), armazenado ****encriptado****)

CustomerAddress

Representa o endereço de um cliente.

- `customer\_address\_id` (PK)

Ano Letivo 2025/26

1ª Fase Relatório Técnico – Complementos de Bases de Dados

- `customer\_id` (FK → Customer)
- `address\_line1` (NVARCHAR(255))
- `city` (NVARCHAR(100))
- `state\_province\_id` (FK → StateProvince)
- `postal\_code` (NVARCHAR(20))
- `country\_id` (FK → CountryRegion)
- `phone` (NVARCHAR(50))

StateProvince

Representa um estado ou província.

- `state\_province\_id` (PK)
- `code` (NVARCHAR(10))
- `name` (NVARCHAR(100))
- `country\_id` (FK → CountryRegion)

CountryRegion

Representa um país ou região.

- `country\_id` (PK)
- `code` (NVARCHAR(10))
- `name` (NVARCHAR(100))

SalesOrder

Representa uma venda (pedido principal).

- `sales\_order\_id` (PK)
- `sales\_order\_number` (NVARCHAR(50))
- `customer\_id` (FK → Customer)
- `sales\_territory\_id` (FK → SalesTerritory)
- `order\_date` (DATE)
- `due\_date` (DATE)
- `currency\_id` (FK → Currency)
- `ship\_date` (DATE)

SalesOrderLine

Representa uma variação de produto vendida dentro de um pedido.

- `sales\_order\_line\_id` (PK)
- `sales\_order\_id` (FK → SalesOrder)
- `line\_number` (INT)
- `product\_variant\_id` (FK → ProductVariant)
- `product\_standard\_cost` (DECIMAL(10,2))
- `unit\_price` (DECIMAL(10,2))
- `quantity` (INT)

1ª Fase Relatório Técnico – Complementos de Bases de Dados

- `tax\_amt` (DECIMAL(10,2))
- `freight` (DECIMAL(10,2))

SalesTerritory

Representa uma região ou território de vendas.

- `sales\_territory\_id` (PK)
- `region` (NVARCHAR(100))
- `country\_region\_id` (FK → CountryRegion)
- `territory\_group` (NVARCHAR(100), opcional)

Currency

Representa a moeda usada nas transações.

- `currency\_id` (PK)
- `code` (NVARCHAR(10))
- `name` (NVARCHAR(50))

AppUser

Representa um utilizador da aplicação (cliente ou administrativo).

- `app\_user\_id` (PK)
- `customer\_id` (FK → Customer, opcional)
- `email` (NVARCHAR(100))
- `password\_hash` (NVARCHAR(255))
- `is\_active` (BIT DEFAULT 1)
- `created\_at` (DATETIME DEFAULT GETDATE())
- `last\_login` (DATETIME)

PasswordRecoveryQuestion

Pergunta de segurança para recuperação de password.

- `question\_id` (PK)
- `app\_user\_id` (FK → AppUser)
- `question\_text` (NVARCHAR(255))
- `answer\_hash` (NVARCHAR(255))

SentEmails

Simula envio de emails.

- `sent\_email\_id` (PK)
- `recipient\_email` (NVARCHAR(100))
- `subject` (NVARCHAR(255))
- `message` (NVARCHAR(MAX))
- `sent\_at` (DATETIME DEFAULT GETDATE())

Ano Letivo 2025/26

Conjuntos de Relacionamentos & Restrições

ProductMaster_ProductVariant (1:N)

- Um **ProductMaster** “possui” várias **ProductVariants**.
- **1 ProductMaster** pode ter **N ProductVariants** **(participação total)**.
- **1 ProductVariant** pertence **sempre** a **1 ProductMaster** **(participação total)**.

ProductMaster_ProductCategory (N:1)

- Um **ProductMaster** “pertence a” uma **ProductCategory**.
- **1 ProductCategory** pode conter **N ProductMasters** **(participação parcial)**.
- **1 ProductMaster** pertence **sempre** a **1 ProductCategory** **(participação total)**.

ProductMaster_ProductSubcategory (N:1)

- Um **ProductMaster** “pertence a” uma **ProductSubcategory**.
- **1 ProductSubcategory** pode ter **N ProductMasters** **(participação parcial)**.
- **1 ProductMaster** pertence **sempre** a **1 ProductSubcategory** **(participação total)**.

ProductMaster_ProductLine (N:1)

- Um **ProductMaster** “tem” uma **ProductLine**.
- **1 ProductLine** pode conter **N ProductMasters** **(participação parcial)**.
- **1 ProductMaster** pertence **sempre** a **1 ProductLine** **(participação total)**.

ProductMaster_ProductClass(N:1)

- Um **ProductMaster** “pertence a” uma **ProductClass**.
- **1 ProductClass** pode ter **N ProductMasters** **(participação parcial)**.
- **1 ProductMaster** pertence **sempre** a **1 ProductClass** **(participação total)**.

ProductVariant_ProductColor (N:1)

- Um **ProductVariant** “tem” uma **ProductColor**.
- **1 ProductColor** pode ser usada por **N ProductVariants** **(participação parcial)**.
- **1 ProductVariant** pode **não ter cor** **(participação parcial)**.

ProductVariant_ProductStyle (N:1)

- Um **ProductVariant** “tem” um **ProductStyle**.
- **1 ProductStyle** pode estar associado a **N ProductVariants** **(participação parcial)**.
- **1 ProductVariant** pode **não ter estilo** **(participação parcial)**.

ProductVariant_ProductSizeRange (N:1)

- Um **ProductVariant** “tem” um **ProductSizeRange**.
- **1 ProductSizeRange** pode estar associado a **N ProductVariants** **(participação parcial)**.
- **1 ProductVariant** pode **não ter SizeRange** **(participação parcial)**.

ProductVariant_UnitOfMeasure (N:1)

- Um **ProductVariant** “utiliza” unidades de medida (peso e tamanho).
- **1 UnitOfMeasure** pode ser usada em **N ProductVariants** **(participação parcial)**.
- **1 ProductVariant** pode **não ter unidade definida** **(participação parcial)**.

Customer_CustomerAddress (1:N)

- Um **Customer** “pode ter” vários **CustomerAddresses**.
- **1 Customer** pode ter **N endereços** **(participação parcial)**.
- **1 CustomerAddress** pertence **sempre** a **1 Customer** **(participação total)**.

CustomerAddress_StateProvince (N:1)

- Um **CustomerAddress** “pertence a” um **StateProvince**.
- **1 StateProvince** pode ter **N CustomerAddresses** **(participação parcial)**.
- **1 CustomerAddress** pertence **sempre** a **1 StateProvince** **(participação total)**.

CustomerAddress_CountryRegion (N:1)

- Um **CustomerAddress** “pertence a” um **CountryRegion**.
- **1 CountryRegion** pode ter **N CustomerAddresses** **(participação parcial)**.
- **1 CustomerAddress** pertence **sempre** a **1 CountryRegion** **(participação total)**.

StateProvince_CountryRegion (N:1)

- Um **StateProvince** “pertence a” um **CountryRegion**.
- **1 CountryRegion** pode ter **N StateProvinces** **(participação parcial)**.
- **1 StateProvince** pertence **sempre** a **1 CountryRegion** **(participação total)**.

SalesOrder_SalesOrderLine (1:N)

1ª Fase Relatório Técnico – Complementos de Bases de Dados

- Um **SalesOrder** “contém” várias **SalesOrderLines**.
- **1 SalesOrder** tem **N SalesOrderLines** **(participação total)**.
- **1 SalesOrderLine** pertence **sempre** a **1 SalesOrder** **(participação total)**.

SalesOrderLine_ProductVariant (N:1)

- Uma **SalesOrderLine** “refere-se a” um **ProductVariant**.
- **1 ProductVariant** pode aparecer em **N SalesOrderLines** **(participação parcial)**.
- **1 SalesOrderLine** refere-se **sempre** a **1 ProductVariant** **(participação total)**.

SalesOrder_Customer (N:1)

- Um **SalesOrder** “é feito por” um **Customer**.
- **1 Customer** pode ter **N SalesOrders** **(participação parcial)**.
- **1 SalesOrder** pertence **sempre** a **1 Customer** **(participação total)**.

SalesOrder_SalesTerritory (N:1)

- Um **SalesOrder** “ocorre em” um **SalesTerritory**.
- **1 SalesTerritory** pode estar associado a **N SalesOrders** **(participação parcial)**.
- **1 SalesOrder** pertence **sempre** a **1 SalesTerritory** **(participação total)**.

SalesOrder_Currency (N:1)

- Uma **SalesOrder** “é faturada em” uma **Currency**.
- **1 Currency** pode estar associada a **N SalesOrders** **(participação parcial)**.
- **1 SalesOrder** usa **sempre 1 Currency** **(participação total)**.

AppUser_Customer (1:1)

- Um **AppUser** “pode estar associado a” um **Customer**.
- **1 Customer** pode ter **no máximo 1 AppUser** **(participação parcial)**.
- **1 AppUser** pode **não ter Customer** **(participação parcial)**.

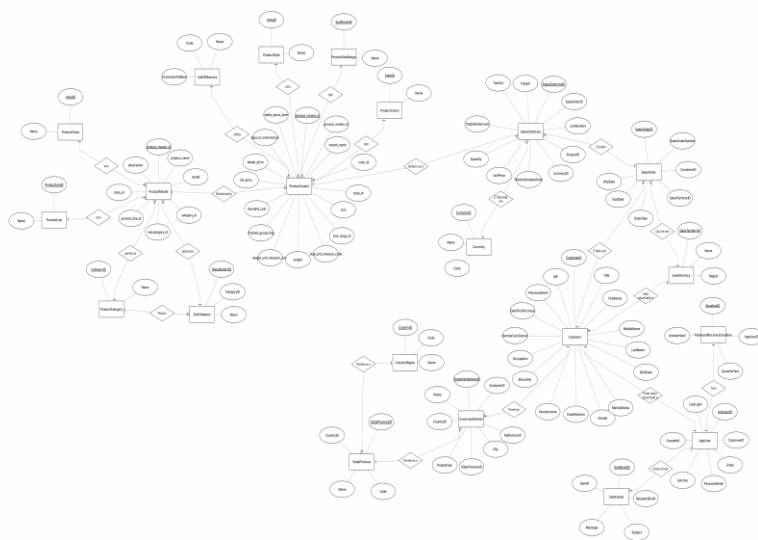
AppUser_PasswordRecoveryQuestion (1:1)

- Um **AppUser** “tem” uma **PasswordRecoveryQuestion**.
- **1 AppUser** tem **no máximo 1 questão** **(participação parcial)**.
- **1 PasswordRecoveryQuestion** pertence **sempre** a **1 AppUser** **(participação total)**.

AppUser_SentEmails (1:N)

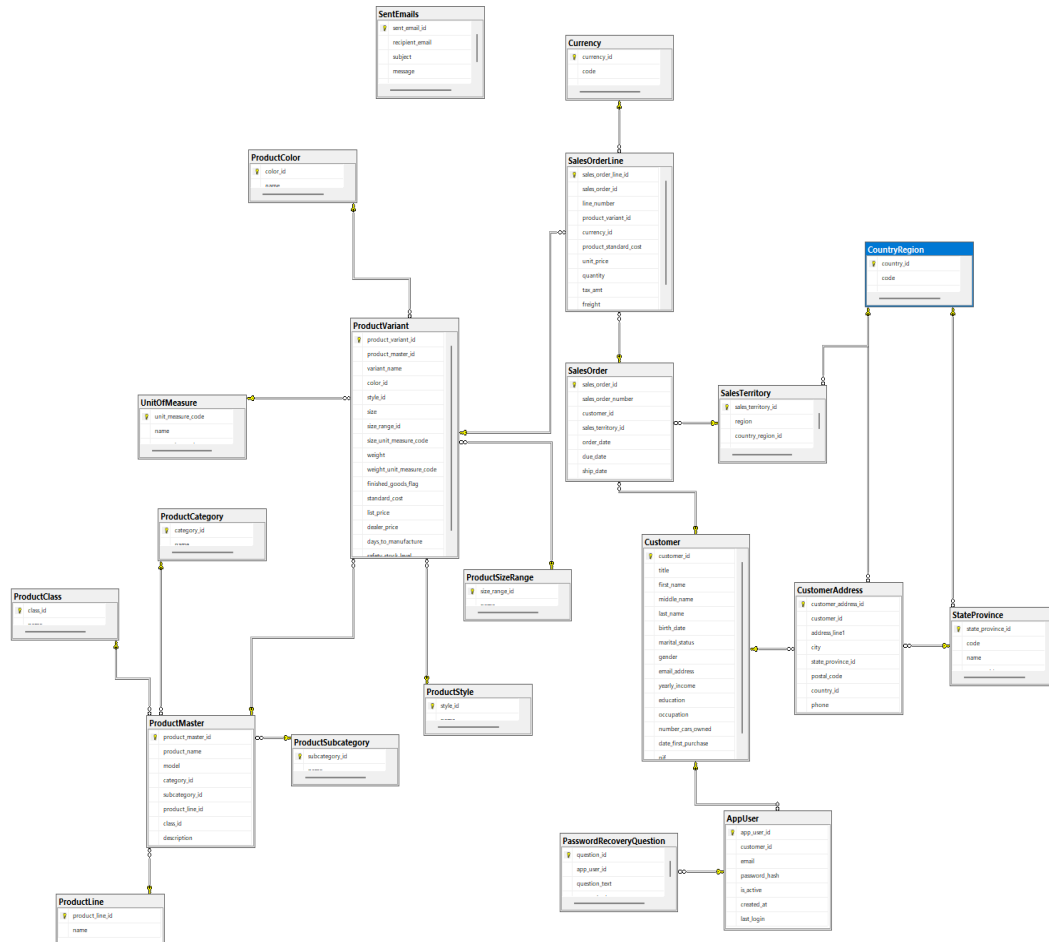
- Um **AppUser** “pode enviar” vários **SentEmails**.
- **1 AppUser** pode ter **N emails enviados** **(participação parcial)**.
- **1 SentEmail** pertence **sempre** a **1 AppUser** **(participação total)**.

3.1. Diagrama do Modelo Entidade Relação



1ª Fase Relatório Técnico – Complementos de Bases de Dados

3.2. Diagrama do Modelo Relacional



4. Definição do Layout

A base de dados AdventureWorks foi estruturada de forma a otimizar o desempenho e a gestão de espaço através da utilização de Filegroups.

Cada filegroup agrega um conjunto de tabelas com características de utilização semelhantes, permitindo isolar operações intensivas de leitura e escrita das tabelas de referência e apoio.

Para suportar esta organização, foi realizada uma análise ao volume de dados e à frequência de acesso após a migração da base de dados, identificando-se as tabelas de uso mais intensivo e as de consulta predominante.

Com base nessa análise, foram definidas prioridades de armazenamento que orientaram a criação dos filegroups, separando as áreas de dados transacionais, referenciais e estruturais de forma a otimizar o desempenho global e a eficiência do sistema.

1ª Fase Relatório Técnico – Complementos de Bases de Dados

4.1. Identificação do espaço ocupado por tabela

Nome Tabela	Dimensão do Registo	Nº de Registos (inicial/final)
SaleOrderLine	67,27	60398
Customer	202,98	18484
AppUser	200,77	18484
CustomerAddress	130,74	18484
SalesOrder	53,02	27659
ProductVariant	165,08	397
ProductMaster	275,36	119
ProductClass	2730,67	3
ProductStyle	2730,67	3
ProductCategory	2048	4
ProductLine	2048	4
UnitOfMeasure	1638,4	5
CountryRegion	1365,33	6
ProductColor	819,2	10
ProductSizeRange	819,2	10
SalesTerritory	819,2	10
dbStatistics	372,36	22
ProductSubcategory	221,41	37

1ª Fase Relatório Técnico – Complementos de Bases de Dados

StateProvince	154,57	53
Currency	78,02	105
PasswordRecoveryQuestion	0	0
SentEmails	0	0

4.2. Especificação dos Filegroups

Nome Filegroup	Tabelas associadas	Parâmetros
FG_Primary	Tabelas de sistema	Dimensão inicial: 10MB Taxa de crescimento: 10%
FG_HighUsage	Tabelas de Escrita/Leitura muito frequente - SaleOrder - SaleOrderLine - ProductVariant - Customer	Dimensão inicial: 8 GB (~6Gb de dados) Taxa de crescimento: (10%)
FG_Secondary	Tabelas associadas: - ProductMaster - Currency - SalesTerritory - ProductCategory - ProductSubcategory - ProductLine - ProductColor - ProductClass - ProductStyle - ProductSizeRange - UnitOfMeasure - CountryRegion - StateProvince - SentEmails	Dimensão inicial: 2 GB (~<1Gb de dados) Taxa de crescimento: (512M)

4.3.Schemas

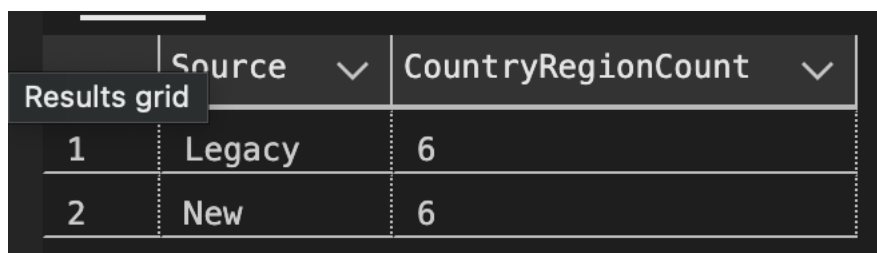
Nome	Descrição
Customer	Reúne a informação associada aos clientes e respetivos dados de contacto.
Sales	Contém todas as entidades relacionadas com o processo de venda e faturação.
Product	Inclui toda a estrutura de produtos e respetivas características.
Core	Armazena tabelas de apoio, lookup e configuração global.

5. Verificação da migração de dados

Abaixo estão os queries utilizados para a validação de dados, após o grupo garantir que a migração de dados estava bem feita foram validados se o número total de records bate certo com a base de dados Legacy.

Nos pontos seguintes estão misturados queries à base de dados antiga assim como à nova em simultâneo de modo a ser fácil validar se existe uma diferença no número de records.

Exemplo:



The screenshot shows a 'Results grid' with two columns: 'Source' and 'CountryRegionCount'. The first row shows 'Legacy' with a count of 6. The second row shows 'New' with a count of 6.

	Source	CountryRegionCount
1	Legacy	6
2	New	6

5.1.Consultas sobre a base de dados original

```
SELECT
    'Legacy' AS Source,
    COUNT(DISTINCT L.CurrencyAlternateKey) AS CurrencyCount
FROM AdventureWorksLegacy.dbo.Currency AS L
UNION ALL
SELECT
    'New',
    COUNT(*)
FROM AdventureWorks.dbo.Currency;
```

Objetivo: Compara o número total de moedas existentes entre a base de dados antiga (AdventureWorksLegacy) e a nova (AdventureWorks).

Results		Messages
	Source	CurrencyCount
1	Legacy	105
2	New	105

1ª Fase Relatório Técnico – Complementos de Bases de Dados

```
SELECT
    'Legacy' AS Source,
    COUNT(DISTINCT L.SalesTerritoryRegion) AS TerritoryCount
FROM AdventureWorksLegacy.dbo.SalesTerritory AS L

UNION ALL

SELECT
    'New',
    COUNT(*)
FROM AdventureWorks.dbo.SalesTerritory;
```

Objetivo: Comparar regiões de território bases antiga e nova.

	Source	TerritoryCount
1	Legacy	11
2	New	10

a quantidade de de vendas entre as

1ª Fase Relatório Técnico – Complementos de Bases de Dados

```
SELECT COUNT(*) AS UnmappedLegacyOrders FROM AdventureWorksLegacy.dbo.Sales AS S LEFT JOIN
AdventureWorks.dbo.SalesOrder AS SO ON dbo.TrimSpaces(S.SalesOrderNumber) =
dbo.TrimSpaces(SO.sales_order_number) WHERE SO.sales_order_id IS NULL;
```

	Source	SalesOrderCount
1	Legacy	27659
2	New	27659

Objetivo: Identificar encomendas da base antiga que não têm correspondência na base nova.

```
SELECT
    'Legacy' AS Source,
    COUNT(*) AS SalesOrderLineCount
FROM AdventureWorksLegacy.dbo.Sales AS S
UNION ALL
SELECT
    'New',
    COUNT(*)
FROM AdventureWorks.dbo.SalesOrderLine;
```

	Source	SalesOrderLineCount
1	Legacy	60398
2	New	60398

Objetivo: Comparar o número total de linhas de encomenda entre a base antiga e a nova.

1ª Fase Relatório Técnico – Complementos de Bases de Dados

```
SELECT
    'Legacy' AS Source,
    COUNT(DISTINCT EnglishProductCategoryName) AS CategoryCount
FROM AdventureWorksLegacy.dbo.Products

UNION ALL

SELECT
    'New',
    COUNT(*) AS CategoryCount
FROM AdventureWorks.dbo.ProductCategory;
```

Objetivo: Comparar o número de categorias de produtos entre a base de dados antiga e a nova.

Results grid	CategoryCount
1 Legacy	4
2 New	4

```
SELECT
    'Legacy' AS Source,
    COUNT(*) AS SubCategoryCount
FROM AdventureWorksLegacy.dbo.ProductSubCategory

UNION ALL

SELECT
    'New',
    COUNT(*) AS SubCategoryCount
FROM AdventureWorks.dbo.ProductSubcategory;
```

1ª Fase Relatório Técnico – Complementos de Bases de Dados

Objetivo: Comparar a quantidade total de subcategorias entre as duas bases.

	Source	SubCategoryCount
1	Legacy	37
2	New	37

```
SELECT
    'Legacy' AS Source,
    COUNT(DISTINCT L.Color) AS ColorCount
FROM AdventureWorksLegacy.dbo.Products AS L
WHERE L.Color IS NOT NULL AND L.Color <> ''
UNION ALL
SELECT
    'New',
    COUNT(*) AS ColorCount
FROM AdventureWorks.dbo.ProductColor;
```

Objetivo: Comparar o número de cores de produtos distintas entre as duas bases.

	Source	ColorCount
1	Legacy	9
2	New	10

1ª Fase Relatório Técnico – Complementos de Bases de Dados

```
SELECT
    'Legacy' AS Source,
    COUNT(DISTINCT dbo.TrimSpaces(L.SizeRange)) AS SizeRangeCount
FROM AdventureWorksLegacy.dbo.Products AS L
WHERE L.SizeRange IS NOT NULL AND L.SizeRange <> ''

UNION ALL

SELECT
    'New',
    COUNT(*) AS SizeRangeCount
FROM AdventureWorks.dbo.ProductSizeRange;
```

Objetivo: Comparar o número de faixas de tamanho (SizeRange) entre as bases.

	Source ▾	SizeRangeCount ▾
1	Legacy	10
2	New	10

1ª Fase Relatório Técnico – Complementos de Bases de Dados

```
SELECT
    'Legacy' AS Source,
    COUNT(DISTINCT dbo.TrimSpaces(L.Class)) AS ProductClassCount
FROM AdventureWorksLegacy.dbo.Products AS L
WHERE L.Class IS NOT NULL AND L.Class <> ''
UNION ALL
SELECT
    'New',
    COUNT(*) AS ProductClassCount
FROM AdventureWorks.dbo.ProductClass;
```

Objetivo: Comparar a quantidade de classes de produtos entre as duas bases.

	Source	ProductClassCount
1	Legacy	3
2	New	3

1ª Fase Relatório Técnico – Complementos de Bases de Dados

SELECT

'Legacy' AS Source,

COUNT(*) AS ProductVariantCount

FROM AdventureWorksLegacy.dbo.Products

UNION ALL

SELECT

'New',

COUNT(*) AS ProductVariantCount

FROM AdventureWorks.dbo.ProductVariant;

Objetivo: Comparar o número total de variantes de produto (combinações de cor, tamanho, estilo, etc.) entre as bases.

	Source ▾	ProductVariantCount ▾
1	Legacy	397
2	New	397

1ª Fase Relatório Técnico – Complementos de Bases de Dados

SELECT

'Legacy' AS Source,

COUNT(DISTINCT dbo.TrimSpaces(C.CountryRegionCode)) AS CountryRegionCount

FROM AdventureWorksLegacy.dbo.Customer AS C

WHERE C.CountryRegionCode IS NOT NULL

UNION ALL

SELECT

'New',

COUNT(*) AS CountryRegionCount

FROM AdventureWorks.dbo.CountryRegion;

	Source	CountryRegionCount
1	Legacy	6
2	New	6

Objetivo: Comparar o número total de regiões de país (CountryRegion) entre a base antiga e a nova.

1ª Fase Relatório Técnico – Complementos de Bases de Dados

```
SELECT
    'Legacy' AS Source,
    COUNT(DISTINCT dbo.TrimSpaces(C.StateProvinceCode)) AS StateProvinceCount
FROM AdventureWorksLegacy.dbo.Customer AS C
WHERE C.StateProvinceCode IS NOT NULL

UNION ALL

SELECT
    'New',
    COUNT(*) AS StateProvinceCount
FROM AdventureWorks.dbo.StateProvince;
```

	Source	StateProvinceCount
1	Legacy	53
2	New	53

Objetivo: Comparar o número total de estados/províncias entre as duas bases de dados.

```
SELECT
    'Legacy' AS Source,
    COUNT(*) AS CustomerCount
FROM AdventureWorksLegacy.dbo.Customer
WHERE EmailAddress IS NOT NULL

UNION ALL

SELECT
    'New',
    COUNT(*) AS CustomerCount
FROM AdventureWorks.dbo.Customer;
```

1ª Fase Relatório Técnico – Complementos de Bases de Dados

Objetivo: Comparar o número total de clientes migrados entre as duas bases.

	Source ▾	CustomerCount ▾
1	Legacy	18484
2	New	18484

1ª Fase Relatório Técnico – Complementos de Bases de Dados

```
SELECT
    'Legacy' AS Source,
    COUNT(*) AS AddressCount
FROM AdventureWorksLegacy.dbo.Customer
WHERE AddressLine1 IS NOT NULL
UNION ALL
SELECT
    'New',
    COUNT(*) AS AddressCount
FROM AdventureWorks.dbo.CustomerAddress;
```

Objetivo: Comparar o número de endereços de clientes entre as duas bases.

	Source	AddressCount
1	Legacy	18484
2	New	18484

5.2.Consultas sobre a nova base de dados

```
SELECT COUNT(*) AS LinesWithoutValidVariant FROM AdventureWorks.dbo.SalesOrderLine AS L LEFT JOIN
AdventureWorks.dbo.ProductVariant AS PV ON L.product_variant_id = PV.product_variant_id WHERE
PV.product_variant_id IS NULL;
```

Objetivo: Detetar linhas de encomenda que não estão associadas a um produto válido.

1ª Fase Relatório Técnico – Complementos de Bases de Dados

```
SELECT TOP 10 S.SalesOrderNumber, S.TotalSalesAmount, S.UnitPrice, CAST( ROUND(
TRY_CONVERT(DECIMAL(18,4), S.TotalSalesAmount) / NULLIF(TRY_CONVERT(DECIMAL(18,4), S.UnitPrice), 0), 0) AS
INT ) AS ExpectedQuantityFromLegacy, L.quantity AS MigratedQuantity FROM AdventureWorksLegacy.dbo.Sales AS
S JOIN AdventureWorks.dbo.SalesOrder AS SO ON dbo.TrimSpaces(S.SalesOrderNumber) =
dbo.TrimSpaces(SO.sales_order_number) JOIN AdventureWorks.dbo.SalesOrderLine AS L ON SO.sales_order_id =
L.sales_order_id ORDER BY S.SalesOrderNumber;
```

Objetivo: Validar se a quantidade migrada (L.quantity) corresponde à quantidade calculada a partir dos valores da base antiga.

```
SELECT COUNT(*) AS OrphanedOrders FROM AdventureWorks.dbo.SalesOrder AS SO LEFT JOIN
AdventureWorks.dbo.Customer AS C ON SO.customer_id = C.customer_id WHERE C.customer_id IS NULL;
```

Objetivo: Detetar encomendas sem cliente associado.

```
SELECT DISTINCT
    L.ModelName
FROM AdventureWorksLegacy.dbo.Products AS L
WHERE L.ModelName NOT IN (
    SELECT PM.model FROM AdventureWorks.dbo.ProductMaster AS PM
);
```

Objetivo: Detetar modelos que existem na base antiga mas não foram migrados para ProductMaster.

```
SELECT COUNT(*) AS OrphanVariants FROM AdventureWorks.dbo.ProductVariant AS PV LEFT JOIN
AdventureWorks.dbo.ProductMaster AS PM ON PV.product_master_id = PM.product_master_id WHERE
PM.product_master_id IS NULL;
```

Objetivo: Verificar a integridade referencial entre ProductVariant e ProductMaster.

1ª Fase Relatório Técnico – Complementos de Bases de Dados

```
SELECT DISTINCT dbo.TrimSpaces(C.CountryRegionCode) AS MissingCountryRegionCode
FROM AdventureWorksLegacy.dbo.Customer AS C
WHERE C.CountryRegionCode IS NOT NULL
AND NOT EXISTS (
    SELECT 1
    FROM AdventureWorks.dbo.CountryRegion AS CR
    WHERE CR.code = dbo.TrimSpaces(C.CountryRegionCode)
);
```

Objetivo: Identificar códigos de país presentes na base antiga mas ausentes na nova.

```
SELECT DISTINCT dbo.TrimSpaces(C.StateProvinceCode) AS MissingStateProvinceCode
FROM AdventureWorksLegacy.dbo.Customer AS C
WHERE C.StateProvinceCode IS NOT NULL
AND NOT EXISTS (
    SELECT 1
    FROM AdventureWorks.dbo.StateProvince AS SP
    WHERE SP.code = dbo.TrimSpaces(C.StateProvinceCode)
);
```

Objetivo: Detetar códigos de estado/província que não foram migrados corretamente.

```
SELECT
    L.EmailAddress AS MissingEmail
FROM AdventureWorksLegacy.dbo.Customer AS L
WHERE L.EmailAddress IS NOT NULL
AND NOT EXISTS (
    SELECT 1
    FROM AdventureWorks.dbo.Customer AS N
    WHERE N.email_address = dbo.TrimSpaces(L.EmailAddress)
```

1ª Fase Relatório Técnico – Complementos de Bases de Dados

);

Objetivo: Identificar clientes que não foram migrados (com base no email).

```
SELECT COUNT(*) AS OrphanCustomerAddresses FROM AdventureWorks.dbo.CustomerAddress AS CA LEFT JOIN
AdventureWorks.dbo.Customer AS C ON CA.customer_id = C.customer_id WHERE C.customer_id IS NULL;
```

Objetivo: Detetar endereços sem cliente associado na nova base de dados.

```
SELECT COUNT(*) AS OrphanedAppUsers FROM AdventureWorks.dbo.AppUser AS AU LEFT JOIN
AdventureWorks.dbo.Customer AS C ON AU.customer_id = C.customer_id WHERE C.customer_id IS NULL;
```

Objetivo: Garantir que todos os utilizadores (AppUser) estão corretamente associados a um cliente (Customer).

```
SELECT TOP 10 C.email_address AS CustomerEmail, AU.email AS AppUserEmail FROM AdventureWorks.dbo.AppUser
AS AU JOIN AdventureWorks.dbo.Customer AS C ON AU.customer_id = C.customer_id WHERE
dbo.TrimSpaces(AU.email) <> dbo.TrimSpaces(C.email_address);
```

Objetivo: Verificar se o email do AppUser coincide com o email do Customer associado.

```
SELECT
    COUNT(*) AS NullHashes
FROM AdventureWorks.dbo.AppUser
WHERE password_hash IS NULL;
```

```
SELECT
    MIN(LEN(password_hash)) AS MinHashLength,
    MAX(LEN(password_hash)) AS MaxHashLength
FROM AdventureWorks.dbo.AppUser;
```

Objetivo: Confirmar que todas as palavras-passe foram corretamente encriptadas e armazenadas em formato de hash.

1ª Fase Relatório Técnico – Complementos de Bases de Dados

Query especifica ao Grupo20 - Top5 productos mais vendidos

153

154

155

156

157

158

159

160

161

162

163

164

```
SELECT TOP 5
  P.EnglishProductName AS ProductName,
  SUM(TRY_CONVERT(DECIMAL(18,4), S.TotalSalesAmount) /
    NULLIF(TRY_CONVERT(DECIMAL(18,4), S.UnitPrice), 0)) AS TotalQuantitySold,
  SUM(TRY_CONVERT(DECIMAL(18,4), S.TotalSalesAmount)) AS TotalSalesAmount
FROM dbo.Sales AS S
INNER JOIN dbo.Products AS P
  ON S.ProductKey = P.ProductKey
GROUP BY P.EnglishProductName
ORDER BY SUM(TRY_CONVERT(DECIMAL(18,4), S.TotalSalesAmount) /
  NULLIF(TRY_CONVERT(DECIMAL(18,4), S.UnitPrice), 0)) DESC;
```

line 158, column 46, location 5166

	ProductName	TotalQuantitySold	TotalSalesAmount	
1	Water Bottle - 30 oz.	710097.19438877755511019910	3543385.0000	
2	Patch Kit/8 Patches	630056.65502183406113535823	1442829.7400	
3	Road Bottle Cage	242533.16573971078976639858	2180373.1600	
4	Road Tire Tube	234805.86967418546365913547	936875.4200	
5	Mountain Tire Tube	208898.11422845691382764124	1042401.5900	

Legacy:

vw_Top5BestSellingProducts				
ProductModel	ProductVariant	TotalQuantitySold	TotalSalesAmount	
Water Bottle	Water Bottle - 30 oz.	710209	3543942.91	
Patch kit	Patch Kit/8 Patches	629890	1442448.10	
Road Bottle Cage	Road Bottle Cage	242725	2182097.75	
Road Tire Tube	Road Tire Tube	234857	937079.43	
Mountain Tire Tube	Mountain Tire Tube	208597	1040899.03	

Nova:

1ª Fase Relatório Técnico – Complementos de Bases de Dados

Validar uma venda e todos productos associados

149
150
151
152
line 150, column 79, location 4923

```
150 SELECT * FROM AdventureWorksLegacy.dbo.Sales WHERE SalesOrderNumber='S075115';
```

	ProductKey	CustomerKey	CurrencyKey	SalesTerritoryKey	SalesOrderNumber	SalesOrderLineNumber	UnitPrice	ProductStandardCost	TotalSalesAmount	TaxAmt	Freight	OrderDate	DueDate	ShipDate
1	538	26832	100	8	S075115	1	21.49	8.04	80.47	1.72	0.54	2014-01-28	2014-02-09	2014-02-04
2	529	26832	100	8	S075115	2	3.99	1.49	80.47	0.32	0.1	2014-01-28	2014-02-09	2014-02-04
3	487	26832	100	8	S075115	3	54.99	20.57	80.47	4.4	1.37	2014-01-28	2014-02-09	2014-02-04

Legacy:

5
6
7
line 6, column 80, location 239

```
6 SELECT * FROM AdventureWorks.dbo.SalesOrder WHERE sales_order_number='S075115';
```

	sales_order_id	sales_order_number	customer_id	sales_territory_id	currency_id	order_date	due_date	ship_date
1	2150	S075115	15833 →	8 →	100 →	2014-01-28	2014-02-09	2014-02-04

Nova:

5
6
7
8
line 5, column 77, location 159

```
5 SELECT * FROM AdventureWorks.dbo.SalesOrderLine WHERE sales_order_id='2150';  
6 SELECT * FROM AdventureWorks.dbo.SalesOrder WHERE sales_order_number='S075115';  
8 SELECT * FROM AdventureWorks.dbo.SalesOrder WHERE customer_id='15833';
```

	sales_order_line_id	sales_order_id	line_number	product_variant_id	product_standard_cost	unit_price	quantity	tax_amt	freight
1	48399	2150 →	1	329 →	8.04	21.49	4	1.72	0.54
2	48686	2150 →	3	278 →	20.57	54.99	1	4.40	1.37
3	49461	2150 →	2	320 →	1.49	3.99	20	0.32	0.10

6. Programação

6.1.Views

Nome	Descrição
<i>dbo.vw_ProductCount</i>	<i>Mostra o número total de produtos (registos em ProductVariant).</i>
dbo.vw_SalesCount	Mostra o número total de encomendas distintas (SalesOrder).
dbo.vw_SalesByCustomer	Apresenta o valor total de vendas por cliente, com base na quantidade e preço unitário.
dbo.vw_SalesByYear	Mostra o total de vendas anuais, agrupando por ano da data de encomenda.
dbo.vw_SalesByYearProduct	Apresenta o valor total de vendas por ano e modelo de produto.
dbo.vw_SalesByRegion	Mostra o total de vendas por região de território (SalesTerritory.region).
dbo.vw_ProductDetailed	Combina as tabelas ProductMaster e ProductVariant para fornecer uma visão detalhada de cada produto, incluindo categoria, cor, estilo e medidas.
dbo.vw_CustomerFullInfo	Fornece informação completa de cada cliente, incluindo dados pessoais, endereço, país, estado, e informação do utilizador (AppUser).
dbo.vw_CustomerSalesSummary	Apresenta um resumo de vendas por cliente, incluindo número de encomendas, linhas, total de vendas, impostos e datas da primeira e última compra.

6.2.Functions

Nome	Atributos	Requisito	Descrição
dbo.fn_check_duplicate_customers()	Sem parametros	R10	Função de verificação de qualidade de dados que devolve uma lista de endereços de e-mail duplicados na tabela Customer. Permite identificar inconsistências resultantes da migração.
dbo.TrimSpaces(@Input NVARCHAR(MAX))	Recebe uma string e remove espaços no início e no fim.	R10	Função auxiliar usada em quase todas as queries de migração para normalizar texto (como nomes de produtos, categorias e e-mails). Garante que dados com espaços extra sejam tratados corretamente.
dbo.ComputeHash(@input NVARCHAR(MAX))	Retorna um valor VARBINARY(32) com o hash SHA2_256 do texto de entrada.	R10	Função utilitária usada para gerar hashes seguros de passwords ou outros campos sensíveis durante a migração, garantindo consistência e confidencialidade.

6.3.Stored procedures

Nome	Atributos	Requisit o	Descrição
dbo.sp_send_email	@recipient_email NVARCHAR(100), @subject NVARCHAR(255), @message NVARCHAR(MAX)	R10	Regista mensagens enviadas na tabela SentEmails. Simula envio de e-mails do sistema.
dbo.sp_add_app_user	@customer_id INT, @email NVARCHAR(100), @password NVARCHAR(255), @question NVARCHAR(255), @answer NVARCHAR(255)	R10	Permite adicionar um novo utilizador (AppUser) associado a um cliente. Inclui criação de pergunta e resposta de recuperação de password com hash.
dbo.sp_update_app_user	@app_user_id INT, @email NVARCHAR(100), @is_active BIT	R10	Atualiza o e-mail ou estado ativo de um utilizador existente.
dbo.sp_delete_app_user	@app_user_id INT	R10	Elimina um utilizador (AppUser) e respetiva pergunta de recuperação de password.

1ª Fase Relatório Técnico – Complementos de Bases de Dados

dbo.sp_authenticate_user	@email NVARCHAR(100), @password NVARCHAR(255)	R10	Autentica um utilizador verificando o e-mail e o hash da password. Atualiza o campo last_login em caso de sucesso.
dbo.sp_recover_password	@email NVARCHAR(100), @answer NVARCHAR(255)	R10	Permite recuperar a password de um utilizador através da resposta de segurança. Gera nova password temporária e envia-a por e-mail simulado.
dbo.sp_list_sent_emails	Sem parametros	R10	Lista todos os e-mails enviados (registados na tabela SentEmails).
dbo.sp_dbstatistics	Sem parametros	R09	Regista na tabela dbStatistics o número de registos, espaço ocupado e data de atualização de todas as tabelas da base de dados. Mantém histórico de execuções sucessivas, permitindo monitorizar o crescimento das tabelas.

6.4.Triggers

Nome	Tipo	Tabela	Requisito	Descrição
dbo.tr_utilizador_historico	AFTER UPDATE	dbo.utilizador	R0#	Guarda o histórico de alterações sobre o utilizador

7. Catálogo/Metadados

7.1.Monitorização

Nome	Atributos	Descrição
dbo.dbStatistics	stat_id INT, table_name NVARCHAR(255), record_count BIGINT, reserved_space_kb DECIMAL(18,2), data_space_kb DECIMAL(18,2), index_size_kb DECIMAL(18,2), unused_space_kb DECIMAL(18,2), collected_at DATETIME	Tabela criada para armazenar estatísticas sobre cada tabela da base de dados, incluindo número de registos, espaço ocupado e data de recolha. Mantém histórico das execuções.
dbo.sp_dbstatistics	(sem parâmetros)	Stored procedure que percorre todas as tabelas da base de dados, recolhe estatísticas com o comando sp_spaceused e regista os resultados na tabela dbStatistics. É usada para monitorizar o crescimento e ocupação de espaço das tabelas.

8. Índices

Legenda:

Q1 – Querie 1 do enunciado no ponto 3.1

Q2 – Querie 2 do enunciado no ponto 3.1

Q3 – Querie 3 do enunciado no ponto 3.1

Querie 1 - Pesquisa de vendas por cidade. Deve ser retornado o nome da cidade, o código do estado e o total de vendas

Querie 2 - Pesquisa de produtos associados a vendas com valor total monetário superior a 1000

Querie 3 - Número de produtos vendidos por categoria por ano.

Índices:

Q1: vendas por cidade/estado

- **City + state_province_id:** agrupa e distingue cidades por estado
- **INCLUDE customer_id:** ajuda joins a partir da morada

```
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name='IX_CustomerAddress_City_State' AND  
object_id=OBJECT_ID('dbo.CustomerAddress'))
```

```
BEGIN
```

```
    CREATE INDEX IX_CustomerAddress_City_State
```

```
    ON dbo.CustomerAddress (city, state_province_id)
```

```
    INCLUDE (customer_id);
```

```
END
```

```
GO
```

Q1/Q3: SalesOrder

- **customer_id:** join ao Customer
- **order_date:** agrupamento por ano (YEAR(order_date))
- **INCLUDE sales_territory_id:** evita lookup quando precisas do território

```
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name='IX_SalesOrder_Customer_OrderDate' AND  
object_id=OBJECT_ID('dbo.SalesOrder'))
```

```
BEGIN
```

```
    CREATE INDEX IX_SalesOrder_Customer_OrderDate  
    ON dbo.SalesOrder (customer_id, order_date)  
    INCLUDE (sales_territory_id);
```

```
END
```

```
GO
```

Q1: SalesOrderLine por sales_order_id

- **acelera join SO->SOL e o cálculo SUM(unit_price*quantity)**
- **INCLUDE** colunas usadas no cálculo e ligação ao produto

```
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name='IX_SalesOrderLine_SalesOrder' AND  
object_id=OBJECT_ID('dbo.SalesOrderLine'))
```

```
BEGIN
```

```
CREATE INDEX IX_SalesOrderLine_SalesOrder
```

```
ON dbo.SalesOrderLine (sales_order_id)
```

```
INCLUDE (unit_price, quantity, product_variant_id);
```

```
END
```

```
GO
```

Q2/Q3: SalesOrderLine por product_variant_id

- **acelera agregações por produto (HAVING > 1000 e por categoria/ano)**
- **INCLUDE sales_order_id** para join rápido ao cabeçalho

```
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name='IX_SalesOrderLine_ProductVariant' AND  
object_id=OBJECT_ID('dbo.SalesOrderLine'))
```

```
BEGIN
```

```
CREATE INDEX IX_SalesOrderLine_ProductVariant
```

```
ON dbo.SalesOrderLine (product_variant_id)
```

```
INCLUDE (unit_price, quantity, sales_order_id);
```

```
END
```

```
GO
```

Querie 1:

```
SELECT
    ca.city,
    sp.code AS state_code,
    SUM(COALESCE(sol.unit_price,0) * COALESCE(sol.quantity,0)) AS total_sales_amount
FROM dbo.SalesOrder so
JOIN dbo.SalesOrderLine sol ON so.sales_order_id = sol.sales_order_id
JOIN dbo.Customer c ON so.customer_id = c.customer_id
JOIN dbo.CustomerAddress ca ON c.customer_id = ca.customer_id
LEFT JOIN dbo.StateProvince sp ON ca.state_province_id = sp.state_province_id
GROUP BY ca.city, sp.code
ORDER BY total_sales_amount DESC;
GO
```

Com Índice:

```
SQL Server Execution Times:
    CPU time = 30 ms,  elapsed time = 55 ms.
```

Sem Índice:

```
SQL Server Execution Times:
    CPU time = 92 ms,  elapsed time = 68 ms.
```

Querie 2:

```
SELECT
    pm.model,
    pv.product_variant_id,
    pv.variant_name,
    SUM(COALESCE(sol.unit_price,0) * COALESCE(sol.quantity,0)) AS total_product_sales
FROM dbo.SalesOrderLine sol
JOIN dbo.ProductVariant pv ON sol.product_variant_id = pv.product_variant_id
JOIN dbo.ProductMaster pm ON pv.product_master_id = pm.product_master_id
GROUP BY pm.model, pv.product_variant_id, pv.variant_name
HAVING SUM(COALESCE(sol.unit_price,0) * COALESCE(sol.quantity,0)) > 1000
ORDER BY total_product_sales DESC;
GO
```

Com Índice:

```
SQL Server Execution Times:
    CPU time = 47 ms,  elapsed time = 77 ms.
```

Sem Índice:

```
SQL Server Execution Times:
    CPU time = 31 ms,  elapsed time = 95 ms.
```

Querie 3:

```
SELECT  
  
    YEAR(so.order_date) AS sales_year,  
  
    pc.name AS category,  
  
    SUM(COALESCE(sol.quantity,0)) AS total_units  
  
FROM dbo.SalesOrderLine sol  
  
JOIN dbo.SalesOrder so    ON sol.sales_order_id = so.sales_order_id  
  
JOIN dbo.ProductVariant pv ON sol.product_variant_id = pv.product_variant_id  
  
JOIN dbo.ProductMaster pm  ON pv.product_master_id = pm.product_master_id  
  
LEFT JOIN dbo.ProductCategory pc ON pm.category_id = pc.category_id  
  
GROUP BY YEAR(so.order_date), pc.name  
  
ORDER BY sales_year, category;  
  
GO
```

Com Índice:

```
SQL Server Execution Times:  
    CPU time = 47 ms,  elapsed time = 38 ms.
```

Sem Índice:

```
SQL Server Execution Times:  
    CPU time = 47 ms,  elapsed time = 46 ms.
```

Selectividade e Densidade

```
SELECT  
COUNT(*) AS total_rows,  
COUNT(DISTINCT CONCAT(city,'|',state_province_id)) AS distinct_city_state,  
CAST(COUNT(DISTINCT CONCAT(city,'|',state_province_id)) AS float)/NULLIF(COUNT(*),0) AS selectivity,  
CAST(COUNT(*) AS float)/NULLIF(COUNT(DISTINCT CONCAT(city,'|',state_province_id)),0) AS density  
FROM dbo.CustomerAddress;
```

Objetivo: Avaliar se a combinação cidade + estado é adequada para indexação em queries que agregam vendas por localização.

Justificação: Cidades com o mesmo nome mas em estados diferentes devem ser consideradas distintas (requisito do enunciado).

	total_rows	distinct_city_state	selectivity	density
1	18484	285	0,0154187405323523	64,8561403508772

```
SELECT  
COUNT(*) AS total_rows,  
COUNT(DISTINCT product_variant_id) AS distinct_products,  
CAST(COUNT(DISTINCT product_variant_id) AS float)/NULLIF(COUNT(*),0) AS selectivity,  
CAST(COUNT(*) AS float)/NULLIF(COUNT(DISTINCT product_variant_id),0) AS density  
FROM dbo.SalesOrderLine;
```

Objetivo: Avaliar a eficácia de indexar o produto associado às vendas, usado em:

- Produtos com vendas > 1000€
- Número de produtos vendidos por categoria

	total_rows	distinct_products	selectivity	density
1	60398	158	0,00261598066161131	382,26582278481

1ª Fase Relatório Técnico – Complementos de Bases de Dados

```
SELECT  
COUNT(*) AS total_rows,  
COUNT(DISTINCT order_date) AS distinct_dates,  
CAST(COUNT(DISTINCT order_date) AS float)/NULLIF(COUNT(*),0) AS selectivity,  
CAST(COUNT(*) AS float)/NULLIF(COUNT(DISTINCT order_date),0) AS density  
FROM dbo.SalesOrder;
```

Objetivo: Avaliar a granularidade temporal das vendas (agrupamento por ano).

	total_rows	distinct_dates	selectivity	density
1	27659	1124	0,0406377670920858	24,6076512455516

9. Backup e Recuperação

1. Modelo de Recuperação

O sistema utiliza o Recovery Model: FULL.

Justificação

- A base de dados contém operações transacionais frequentes (produtos, vendas, criação de utilizadores, etc.).
- É necessário garantir a possibilidade de recuperação ponto-no-tempo.
- Permite restaurar após falhas críticas sem perda de dados, desde que o transaction log seja gerido com backups periódicos.

2. Tipo de backup

Carga no sistema e fatores:

A base de dados suporta um sistema de vendas, o que implica um volume constante de operações de escrita ao longo do dia: criação de clientes, registo de vendas, atualizações de stock, etc... Estes dados têm impacto direto na atividade diária da empresa, pelo que a sua perda não é aceitável. Além disso, o período de maior utilização concentra-se durante o dia, o que limita a execução de operações pesadas, como backups completos, sem afetar o desempenho do sistema.

Tendo isto em conta, é necessário definir um plano que garanta dois objetivos: minimizar a perda de dados e reduzir o tempo de “downtime” em caso de falha. Para tal, opta-se por uma estratégia baseada no modelo de recuperação FULL, que permite recuperar a base de dados até ao ponto exato da falha. Para assegurar que as alterações são registadas com frequência suficiente, os backups do transaction log são executados em intervalos regulares ao longo do dia. Assim, mesmo que ocorra um crash, a perda de informação limita-se aos últimos minutos de atividade.

1ª Fase Relatório Técnico – Complementos de Bases de Dados

Tipos de backup a implementar:

1. Backup Integral (Full Backup) - Semanal

O backup integral copia toda a base de dados num único ficheiro, garantindo um ponto de recuperação completo.

- Executar o full durante a noite evita impacto no desempenho.
- Mantém uma base sólida para restores rápidos.
- Uma periodicidade semanal é suficiente dado o suporte oferecido pelos diferenciais e logs durante a semana.

Calendarização:

- Domingo, 02:00 (período de menor carga)

2. Backup Diferencial – Diário

O backup diferencial regista todas as alterações desde o último full backup.

- Permite recuperar rapidamente o estado da BD sem aplicar todos os logs desde o último full.
- É mais rápido e leve que um full, adequado para a rotina diária..
- Minimiza o impacto no sistema, garantindo uma cópia atualizada sem sobrecarga.

Calendarização:

- Todos os dias às 02:00, exceto domingo.

3. Backup do Transaction Log – Frequente ao longo do dia

O backup do log guarda todas as transações registadas desde o último log backup.

- Evita perda de dados caso a falha ocorra durante o horário comercial.
- Evita perda de dados caso a falha ocorra durante o horário comercial.O que
- Permite recuperação point-in-time.

Calendarização:

Ano Letivo 2025/26

1ª Fase Relatório Técnico – Complementos de Bases de Dados

- A cada 15 minutos (intervalo típico para sistemas transacionais de vendas)

Dispositivos de armazenamento

Para garantir a disponibilidade e integridade dos backups, adotamos a política 3-2-1 aplicada aos dispositivos de armazenamento:

1. Três cópias dos dados:

- Uma cópia principal no disco local, utilizada para restaurações rápidas em caso de falha do sistema.
- Uma cópia em disco de rede ou storage compartilhado, protegendo contra falha do servidor físico.
- Uma cópia off-site (nuvem ou outro data center) para recuperação em caso de desastre total no site principal.

2. Tipos de media diferentes:

- Disco rígido local para backups incrementais e logs frequentes.
- Fita ou storage de rede para backups completos semanais, garantindo maior durabilidade e portabilidade.

Artigo sobre a estratégia: <https://www.backblaze.com/blog/the-3-2-1-backup-strategy/>

Simulação de crash e recuperação de base de dados

Simular a ocorrência de um crash da base de dados, demonstrar perda de dados após falha, e apresentar a sequência correta de recuperação usando:

- Backup FULL
- Backup Diferencial
- Backup TRANSACTION LOG

Pressupostos:

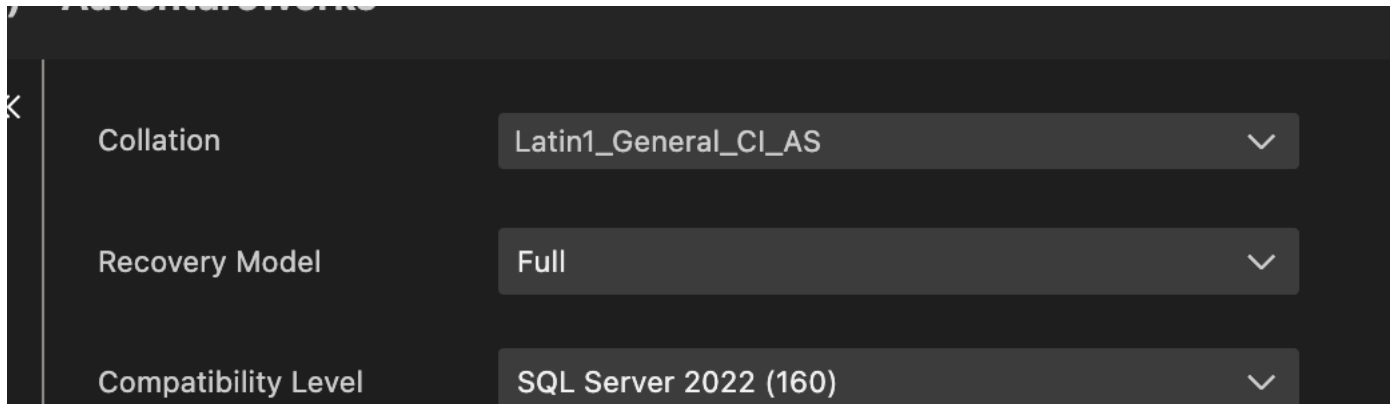
- Base de dados: AdventureWorks
- Modelo de recuperação: FULL
- Backups armazenados em:
 - SQL Server em docker: “/var/opt/mssql/backups/”
 - Windows: “<PASTA-MYSQL>/backups”

Ano Letivo 2025/26

1ª Fase Relatório Técnico – Complementos de Bases de Dados

- Tabela de teste: dbo.ProductCategory
- O “crash” será simulado através de corrupção lógica (ficheiro MDF renomeado)

Simulação de “Crash”



1. Garantir que o modelo de recuperação é “FULL”

2. Fazer um backup full da base de dados

“”””

```
BACKUP DATABASE AdventureWorks
```

```
TO DISK = '/var/opt/mssql/backups/AdventureWorks_FULL.bak'
```

```
WITH INIT, FORMAT;
```

```
GO
```

“”””

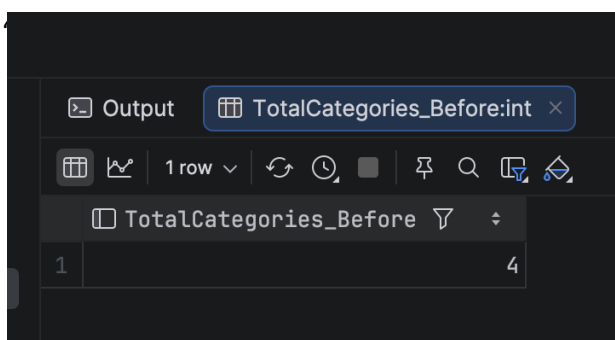
3. Validar estado inicial dos dados

“”””

```
SELECT COUNT(*) AS TotalCategories
```

```
FROM dbo.ProductCategory;
```

```
GO
```



4. Inserir novos dados (após FULL)

/*

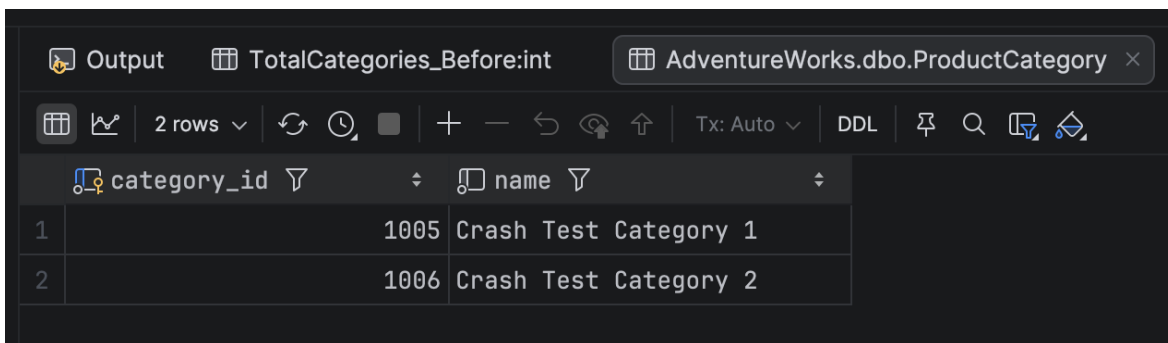
```
INSERT INTO dbo.ProductCategory (name)
```

```
VALUES ('Crash Test Category 1'),
```

```
      ('Crash Test Category 2');
```

```
GO
```

*/



The screenshot shows the SQL Server Enterprise Manager interface. The 'Output' pane is active, displaying the results of a query. The table 'AdventureWorks.dbo.ProductCategory' is shown with 2 rows. The columns are 'category_id' and 'name'.

	category_id	name
1	1005	Crash Test Category 1
2	1006	Crash Test Category 2

5. Validar dados após inserir

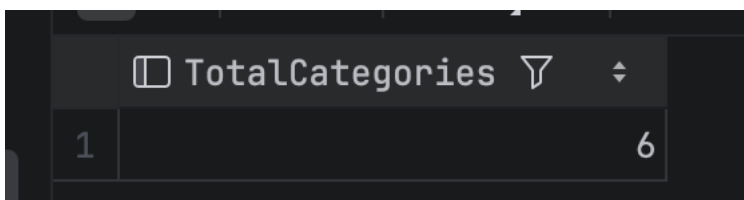
/*

```
SELECT COUNT(*) AS TotalCategories
```

```
FROM dbo.ProductCategory;
```

```
GO
```

*/



The screenshot shows the SQL Server Enterprise Manager interface. The 'Output' pane is active, displaying the results of a query. The table 'TotalCategories' is shown with 1 row. The column is 'TotalCategories'.

	TotalCategories
1	6

6. Backup DIFERENCIAL

"""

USE master;

GO

BACKUP DATABASE AdventureWorks

TO DISK = '/var/opt/mssql/backups/AdventureWorks_DIFF.bak'

WITH DIFFERENTIAL, INIT;

GO

"""

Inserir dados

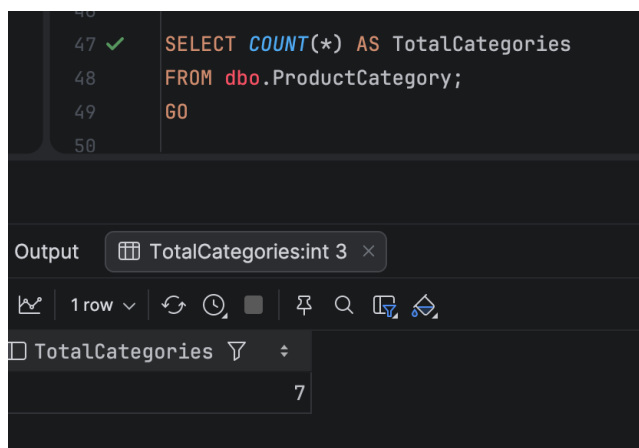
"""

INSERT INTO dbo.ProductCategory (Name)

VALUES ('DIFF BACKUP');

GO

"""



The screenshot shows a SQL query window with the following text:

```
47 ✓ SELECT COUNT(*) AS TotalCategories
48 FROM dbo.ProductCategory;
49 GO
50
```

Below the query window, the 'Output' pane is visible, showing the results of the query:

TotalCategories
7

7. BACKUP DO TRANSACTION LOG

""

USE master;

GO

BACKUP LOG AdventureWorks

TO DISK = '/var/opt/mssql/backups/AdventureWorks_LOG_1.trn'

WITH INIT;

GO

""

8. Inserir após backup Log

""

USE AdventureWorks;

GO

INSERT INTO dbo.ProductCategory (Name)

VALUES ('Protected By Log Backup');

GO

1ª Fase Relatório Técnico – Complementos de Bases de Dados

The screenshot displays the SQL Server Enterprise Manager interface. At the top, a query window shows the following SQL code:

```
142 60
143
144 SELECT *
145 FROM dbo.ProductCategory
146 WHERE Name LIKE '%Lost%';
147 60
148
```

Below the query window, the 'Output' pane is active, showing the results of the query. The results are displayed in a table with the following columns and rows:

	name
1	Accessories
2	Bikes
3	Clothing
4	Components
5	Crash Test Category 1
6	Crash Test Category 2
7	DIFF BACKUP
8	Protected By Log Backup

9. Criar um novo backup log

""

USE master;

GO

BACKUP LOG AdventureWorks

TO DISK = '/var/opt/mssql/backups/AdventureWorks_LOG2.trn'

WITH INIT;

GO

""

10. Inserir dados sem backup (Perda de dados)

""

USE AdventureWorks;

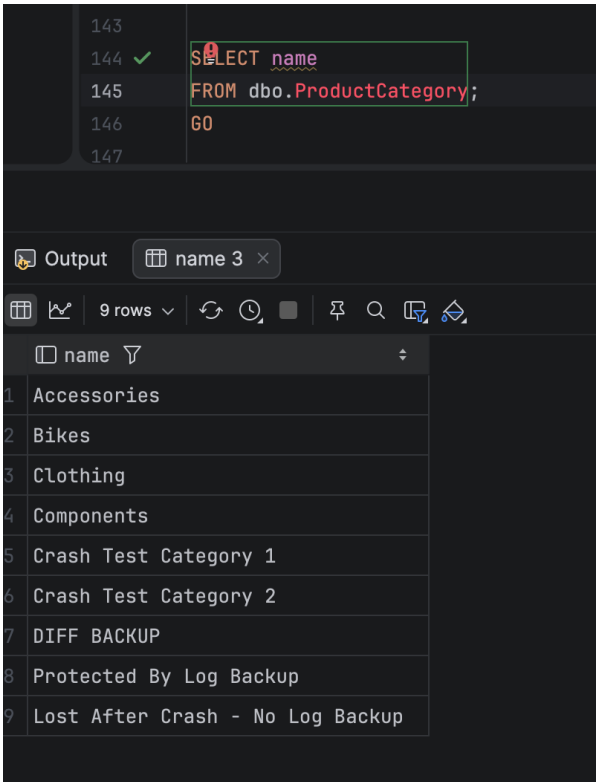
GO

INSERT INTO dbo.ProductCategory (Name)

VALUES ('Lost After Crash - No Log Backup');

GO

""



11. Simular “Crash”

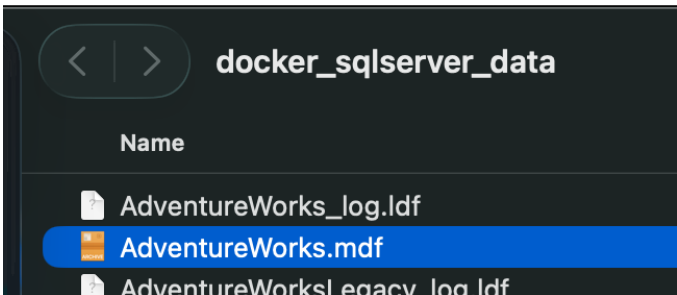
11.1 Desligar servidor de base de dados

No caso do docker, desligar o container.

11.2 Corromper ficheiro de dados do primary filegroup

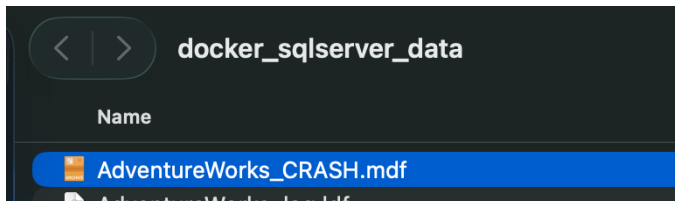
Alterar nome do ficheiro AdventureWorks.mdf → AdventureWorks_CRASH.mdf

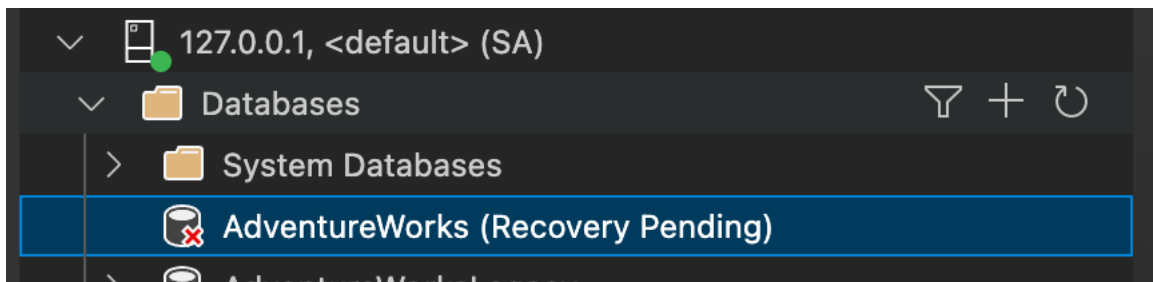
No caso do docker, abrir mount no filesystem e alterar o nome:



1ª Fase Relatório Técnico – Complementos de Bases de Dados

Mudar nome:





12. Ligar base de dados e verificar ausência de informação.

SEQUÊNCIA DE RECUPERAÇÃO

13. Restaurar BACKUP FULL

"""

RESTORE DATABASE AdventureWorks

FROM DISK = '/var/opt/mssql/backups/AdventureWorks_FULL.bak'

WITH

REPLACE,

NORECOVERY; -- Util para quando ainda vao existir mais restores

GO

```
[2025-12-14 18:31:04] completed in 5 ms
[2025-12-14 18:31:06] master> RESTORE DATABASE AdventureWorks
                        FROM DISK = '/var/opt/mssql/backups/AdventureWorks_FULL.bak'
                        WITH
                          REPLACE,
                          NORECOVERY
[2025-12-14 18:31:07] [S0001][4035] Processed 2904 pages for database 'AdventureWorks', file 'AdventureWorks' on file 1.
[2025-12-14 18:31:07] [S0001][4035] Processed 2 pages for database 'AdventureWorks', file 'AdventureWorks_log' on file 1.
[2025-12-14 18:31:07] [S0001][3014] RESTORE DATABASE successfully processed 2906 pages in 0.138 seconds (164.487 MB/sec).
[2025-12-14 18:31:07] completed in 471 ms
```

"""

Ano Letivo 2025/26

14. Restaurar BACKUP DIFFERENTIAL

"""

```
RESTORE DATABASE AdventureWorks_Sales
FROM DISK = 'C:\Backups\AWS_DIFF.bak'
WITH RECOVERY;
GO
```

```
[2025-12-14 18:31:07] completed in 471 ms
[2025-12-14 18:31:33] master> RESTORE DATABASE AdventureWorks
                        FROM DISK = '/var/opt/mssql/backups/AdventureWorks_DIFF.bak'
                        WITH NORECOVERY
[2025-12-14 18:31:33] [S0001][4035] Processed 120 pages for database 'AdventureWorks', file 'AdventureWorks' on file 1.
[2025-12-14 18:31:33] [S0001][4035] Processed 2 pages for database 'AdventureWorks', file 'AdventureWorks_log' on file 1.
[2025-12-14 18:31:33] [S0001][3014] RESTORE DATABASE successfully processed 122 pages in 0.021 seconds (45.200 MB/sec).
[2025-12-14 18:31:33] completed in 284 ms
```

"""

15. Restauro do TRANSACTION LOG

"""

```
RESTORE LOG AdventureWorks
FROM DISK = '/var/opt/mssql/backups/AdventureWorks_LOG.trn'
WITH NORECOVERY;
GO
```

```
[2025-12-14 18:31:33] completed in 284 ms
[2025-12-14 18:32:02] master> RESTORE LOG AdventureWorks
                        FROM DISK = '/var/opt/mssql/backups/AdventureWorks_LOG.trn'
                        WITH NORECOVERY
[2025-12-14 18:32:02] [S0001][4035] Processed 0 pages for database 'AdventureWorks', file 'AdventureWorks' on file 1.
[2025-12-14 18:32:02] [S0001][4035] Processed 22 pages for database 'AdventureWorks', file 'AdventureWorks_log' on file 1.
[2025-12-14 18:32:02] [S0001][3014] RESTORE LOG successfully processed 22 pages in 0.009 seconds (19.097 MB/sec).
[2025-12-14 18:32:02] completed in 187 ms
```

"""

16. Restauro do TRANSACTION LOG 2

"""

```
RESTORE LOG AdventureWorks
FROM DISK = '/var/opt/mssql/backups/AdventureWorks_LOG2.trn'
WITH RECOVERY;

GO
```

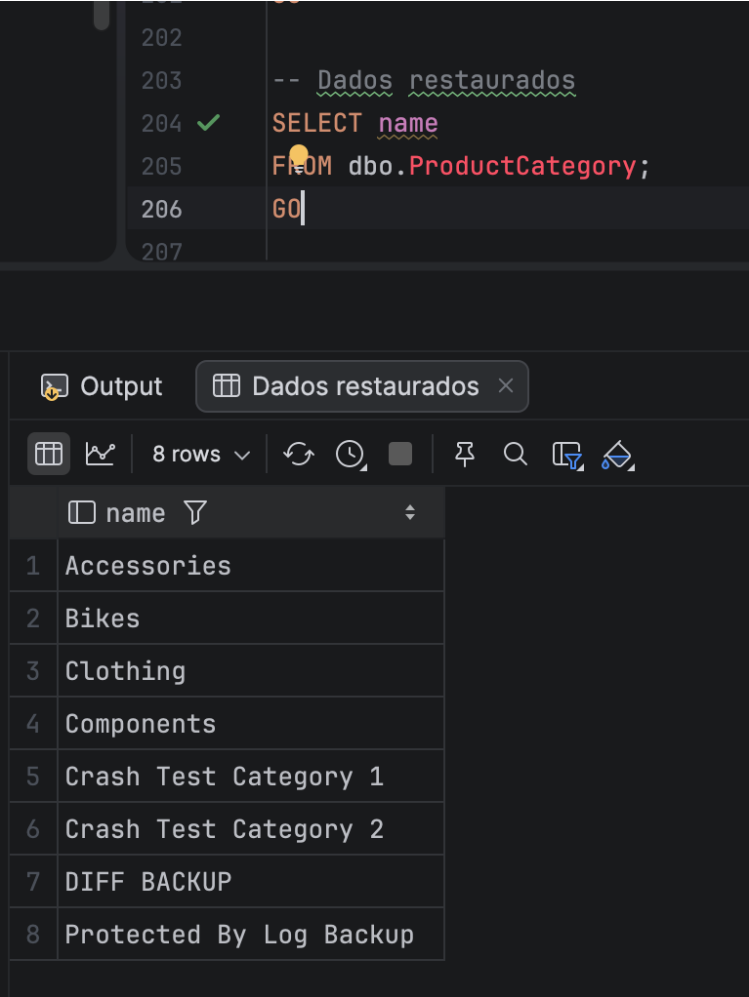
```
[2025-12-14 18:33:59] master> RESTORE LOG AdventureWorks
FROM DISK = '/var/opt/mssql/backups/AdventureWorks_LOG2.trn'
WITH RECOVERY
[2025-12-14 18:34:00] [S0001][4035] Processed 0 pages for database 'AdventureWorks', file 'AdventureWorks' on file 1.
[2025-12-14 18:34:00] [S0001][4035] Processed 2 pages for database 'AdventureWorks', file 'AdventureWorks_log' on file 1.
[2025-12-14 18:34:00] [S0001][3014] RESTORE LOG successfully processed 2 pages in 0.009 seconds (1.736 MB/sec).
[2025-12-14 18:34:00] completed in 661 ms
```

"""

17. Validar dados

```
""
SELECT *
FROM dbo.ProductCategory
WHERE name LIKE '%Test%';
GO
""
```

Perdeu-se o record inserido no passo 9



10. Níveis de Acesso à Informação

Este capítulo define os objetos, as contas de acesso, e os privilégios necessários para suportar os três perfis de utilizador: Administrador, SalesPerson, e SalesTerritory.

As regras foram implementadas utilizando princípios de mínimo privilégio, roles dedicadas, e views auxiliares quando necessário.

1. Administrador

O Administrador necessita de acesso completo a todos os objetos da base de dados.

Objetos/Configuração:

- Criação de uma conta própria (adminUser).
- Criação de um role para agregação de permissões: DBAdministrator.

1ª Fase Relatório Técnico – Complementos de Bases de Dados

Privilégios:

- Membro de db_owner (controlo total sobre a base de dados).
- Permissões:
 - Totais

2. SalesPerson

Utilizador com acesso total a recursos relacionados com vendas e acesso limitado, apenas leitura, dos restantes recursos.

Este utilizador necessita de:

- Acesso total (R/W) às tabelas relacionadas com vendas
- Acesso apenas de leitura às restantes tabelas

Identificação das tabelas de suporte a vendas:

- SalesOrder
- SalesOrderLine
- SalesTerritory

Objetos / Configuração:

- Criação de um role dedicado: roleSalesPerson
- Criação da conta de utilizador: SalesPersonUser

Permissões

- Sobre tabelas de vendas: SELECT, INSERT, UPDATE, DELETE
- Sobre restantes tabelas: SELECT

3. SalesTerritory

O utilizador deste perfil tem acesso exclusivamente à informação relativa ao seu território.

No caso em análise, o território definido é “Southwest”.

(No enunciado estava descrito “RockyMountain”, visto que não existe no nosso Dataset optámos por um outro valor)

Para garantir isolamento de dados e cumprimento do princípio do mínimo privilégio, o acesso é fornecido apenas através de views filtradas. O utilizador não tem acesso direto às tabelas base.

Abordagem

- Criar um role dedicado apenas a leitura: roleSalesTerritory
- Criar o utilizador: SalesTerritoryUser
- Atribuir permissão SELECT somente sobre as views
- Criar um conjunto de views filtradas pelo território “Southwest”, cobrindo:
 - Encomendas (vw_Sales_Southwest)
 - Linhas de vendas (vw_SalesLines_Southwest)
 - Clientes (vw_Customers_Southwest)
 - Produtos vendidos (vw_Products_Southwest)
 - Sumários agregados (vw_SalesSummary_Southwest)

11. Encriptação

Objetivo:

Proteger os dados sensíveis do sistema, garantindo confidencialidade e integridade, aplicando hash para dados irreversíveis e encriptação simétrica para dados que precisam ser recuperáveis.

Campos Sensíveis

O nosso grupo teve como campos sensíveis extras o NIF dos Customers.

A tabela abaixo representa a implementação de todos os campos protegidos e estratégias de proteção utilizadas.

Campo	Tipo de proteção	Método Implementado	Justificação
password	Hash irreversível	SHA2_256 via função <code>dbo.ComputeHash</code>	Hash irreversível garante que mesmo se a BD for comprometida, as passwords não podem ser revertidas. SHA2_256 é robusto e amplamente recomendado para armazenamento seguro de passwords.
NIF	Encriptação simétrica	AES_256 via SYMMETRIC KEY KeyNIF e certificado CertNIF	NIF é um dado sensível que precisa ser recuperável para consultas e relatórios. A encriptação simétrica permite decifrar os valores de forma segura, usando uma chave protegida por certificado.

1ª Fase Relatório Técnico – Complementos de Bases de Dados

Campo	Tipo de proteção	Método Implementado	Justificação
Respostas de recuperação de password	Hash irreversível	SHA2_256 via função dbo.ComputeHash	Respostas de recuperação podem ser exploradas por atacantes. O hash irreversível protege o conteúdo, garantindo que apenas comparações podem ser feitas, sem revelar o valor original.

1ª Fase Relatório Técnico – Complementos de Bases de Dados

Fluxo de Segurança

- **Passwords:** Hash irreversível com SHA2_256 → não é possível recuperar, apenas validar.
- **NIF:** Encriptação simétrica com AES_256 → pode ser recuperado via DECRYPTBYKEY quando necessário.
- **Respostas de recuperação de password:** Hash irreversível → protege contra engenharia reversa de respostas de recuperação.

Consulta de NIF Encriptado e Desencriptado:

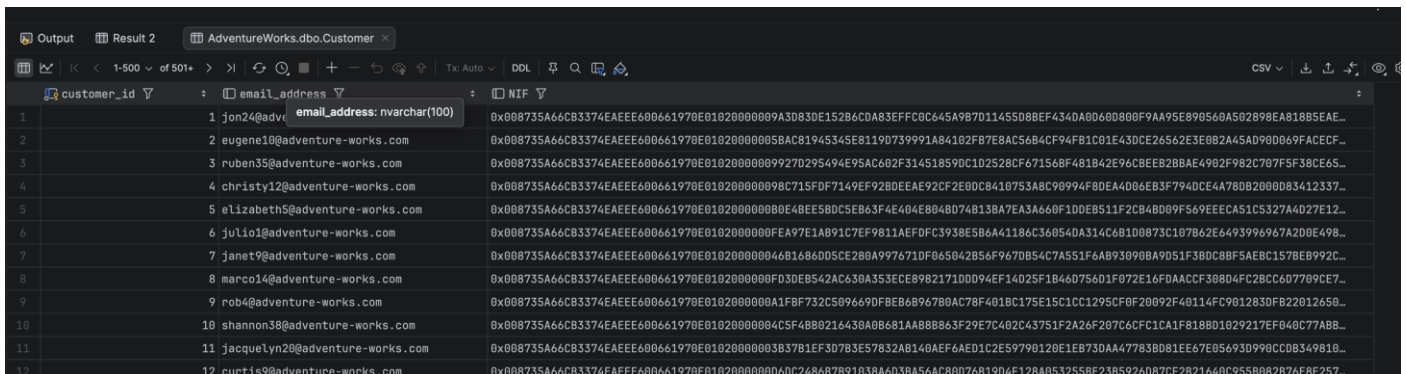
Encriptado

“””

```
SELECT
    customer_id,
    email_address,
    nif AS NIF -- Aqui vemos o valor encriptado (VARBINARY)

FROM dbo.Customer;

GO
```



customer_id	email_address	NIF
1	jon24@adventure-works.com	0x008735A66CB3374EAE6600661970E01020000009A3083DE152B6CDA83EFFF0C645A987D1145508BEF4340A00600800F9AA95E890560A50289EAB18B5EAE...
2	eugene10@adventure-works.com	0x008735A66CB3374EAE6600661970E010200000058ACB1945345E8119D739991A84102F87E8AC5684CF94F81C01E43DCE26562E3E0B2A45AD9D0069FACECF...
3	ruben35@adventure-works.com	0x008735A66CB3374EAE6600661970E01020000009927D295494E95AC602F31451859DC102528CF671568F481B42E96CBE8B2BBAE4902F982C707F5F38CE65...
4	christy12@adventure-works.com	0x008735A66CB3374EAE6600661970E010200000098C715FDF7149EF92B0EEAE92CF2E00C8410753A8C90994F80EA4D06EB3F794DCE4A780B2000D83412337...
5	elizabeth5@adventure-works.com	0x008735A66CB3374EAE6600661970E01020000000E4BE58DC5E863F4E404E8048D74813BA7EA3A660F10DEB511F2C848009F569EEECAS1CS327A4027E12...
6	julio1@adventure-works.com	0x008735A66CB3374EAE6600661970E0102000000FEA97E1AB91C7EF9811AEFDFC3938E586A41186C360540A314C681D0873C107B62E6493996967A2D0E498...
7	janet9@adventure-works.com	0x008735A66CB3374EAE6600661970E010200000046B1684D05CE280A997671DF045042B56F967D0B54C7A551F6AB93090BA9D51F3BDC8BF5AEB1578EB992C...
8	marco14@adventure-works.com	0x008735A66CB3374EAE6600661970E0102000000F3DEB542AC630A353ECE8982171D0D94EF14025F1B460756D1F072E16FDAACCF308D4FC2BCC6D7709CE7...
9	rob4@adventure-works.com	0x008735A66CB3374EAE6600661970E0102000000A1F8F732C599669DF8EB6B967B0AC78F401BC175E15C1CC1295CF0F20092F40114FC901283DFB22012650...
10	shannon38@adventure-works.com	0x008735A66CB3374EAE6600661970E01020000004C5F4B0216430A0B681AAB8863F29E7C402C43751F2A26F207C6CFC1CA1F818BD1029217EF040C77ABB...
11	jacquelyn20@adventure-works.com	0x008735A66CB3374EAE6600661970E01020000003B37B1EF3D783E57832AB140AEF6AED1C2E59790120E1EB730AA47783B081EE67E05693D990CC0B349810...
12	curtis9@adventure-works.com	0x008735A66CB3374EAE6600661970E0102000000DC2486B7B91038A603BA56AC80D76819D4F128A053255BE23B5926D87CF2821640C955B082876E8E257...

“””

Ver dados descriptados

""

OPEN SYMMETRIC KEY KeyNIF

DECRYPTION BY CERTIFICATE CertNIF;

GO

SELECT

customer_id,

email_address,

CONVERT(NVARCHAR(20), DECRYPTBYKEY(nif)) AS NIF -- Valor original em NVARCHAR

FROM dbo.Customer;

GO

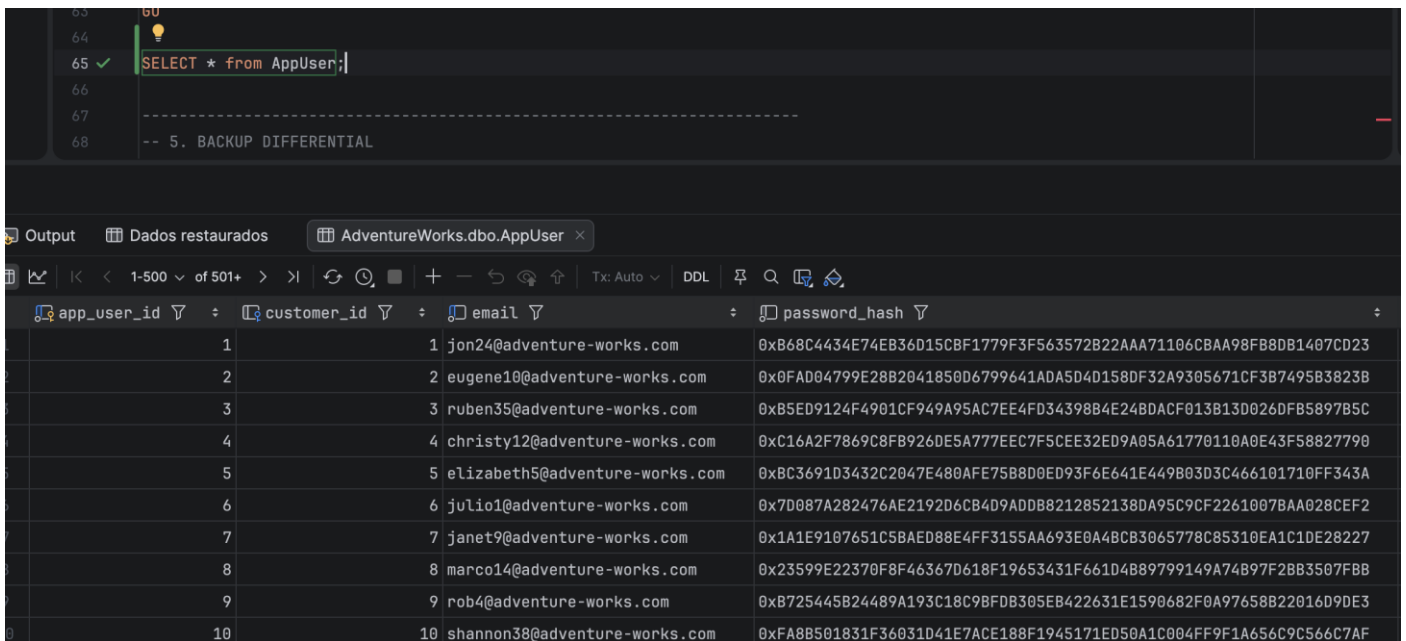
CLOSE SYMMETRIC KEY KeyNIF;

""

1ª Fase Relatório Técnico – Complementos de Bases de Dados

Output			
Result 2			
AdventureWorks.dbo.Customer			
Result 4 2			
1-500 of 501+			
customer_id email_address NIF			
1	1	jon24@adventure-works.com	269192666
2	2	eugene10@adventure-works.com	204599188
3	3	ruben35@adventure-works.com	208688279
4	4	christy12@adventure-works.com	287247418
5	5	elizabeth5@adventure-works.com	240116594
6	6	julio1@adventure-works.com	269839643
7	7	janet9@adventure-works.com	234849397
8	8	marco14@adventure-works.com	282133949
9	9	rob4@adventure-works.com	251664058
10	10	shannon38@adventure-works.com	231888483
11	11	jacquelyn20@adventure-works.com	289454209
12	12	curtis9@adventure-works.com	237947183

1ª Fase Relatório Técnico – Complementos de Bases de Dados



The screenshot shows a SQL Server Enterprise Manager interface. The top pane displays a query: `SELECT * from AppUser;`. The bottom pane shows the results of this query, which is a table with four columns: `app_user_id`, `customer_id`, `email`, and `password_hash`. The results are displayed in a grid format with 10 rows of data.

app_user_id	customer_id	email	password_hash
1	1	jon24@adventure-works.com	0xB68C4434E74EB36D15CBF1779F3F563572B22AAA71106CBAA98FB8DB1407CD23
2	2	eugene10@adventure-works.com	0x0FAD04799E28B2041850D6799641ADA5D4D158DF32A9305671CF3B7495B3823B
3	3	ruben35@adventure-works.com	0xB5ED9124F4901CF949A95AC7EE4FD34398B4E24BDACF013B13D026DFB5897B5C
4	4	christy12@adventure-works.com	0xC16A2F7869C8FB926DE5A777EEC7F5CEE32ED9A05A61770110A0E43F58827790
5	5	elizabeth5@adventure-works.com	0xBC3691D3432C2047E480AFE75B8D0ED93F6E641E449B03D3C466101710FF343A
6	6	julio1@adventure-works.com	0x7D087A282476AE2192D6CB4D9ADD8212852138DA95C9CF2261007BAA028CEF2
7	7	janet9@adventure-works.com	0x1A1E9107651C5BAED88E4FF3155AA693E0A4BCB3065778C85310EA1C1DE28227
8	8	marco14@adventure-works.com	0x23599E22370F8F46367D618F19653431F661D4B89799149A74B97F2BB3507FBB
9	9	rob4@adventure-works.com	0xB725445B24489A193C18C9BFD8305EB422631E1590682F0A97658B22016D9DE3
10	10	shannon38@adventure-works.com	0xFA8B501831F36031D41E7ACE188F1945171ED50A1C004FF9F1A656C9C566C7AF

Consulta password hash

12. Controlo de Transações

Cenário de Conflito – Consulta e Reserva de Stock

Consulta de Stock

No contexto do sistema desenvolvido, o principal cenário de conflito ocorre quando diferentes utilizadores tentam efetuar compras simultâneas do mesmo produto, existindo uma quantidade limitada de stock disponível.

No modelo adotado, o stock disponível não é armazenado num campo explícito, sendo representado pela quantidade de registos existentes na tabela ProductVariant. Cada registo corresponde a uma unidade física disponível para venda. Desta forma, o número de variantes disponíveis determina diretamente o stock do produto.

Este tipo de concorrência pode originar inconsistências, nomeadamente:

- Venda de mais unidades do que as efetivamente disponíveis;
- Consumo do stock abaixo do nível mínimo aceitável (safety stock level);
- Perda de integridade dos dados de vendas e do inventário.

Tabelas envolvidas

O conflito analisado está relacionado com a concorrência na criação de linhas de encomenda (SalesOrderLine) para produtos com stock limitado.

As tabelas envolvidas são:

- ProductVariant – representa as unidades físicas disponíveis em stock;
- SalesOrder – representa a encomenda efetuada pelo cliente;
- SalesOrderLine – representa os produtos vendidos numa encomenda.

Descrição do Problema de Concorrência

O problema surge quando duas transações:

1. Consultam simultaneamente o número de variantes disponíveis para um determinado produto;
2. Assumem que existe stock suficiente para satisfazer o pedido;
3. Tentam reservar ou consumir variantes em paralelo;
4. Executam operações de venda sem coordenação transacional adequada.

Estratégia de Controlo de Concorrência

Abordagem Escolhida

Foi adotada uma estratégia de bloqueio pessimista (pessimistic locking), recorrendo ao nível de isolamento `SERIALIZABLE`.

Esta abordagem garante que, durante o processo de validação e reserva do stock, nenhuma outra transação consegue interferir ou aceder aos mesmos registos de forma concorrente.

Justificação da Escolha

- As operações de venda são críticas e não podem resultar em inconsistências;
- O stock é um recurso limitado, representado por um número finito de registos;
- A integridade dos dados foi considerada prioritária face à performance;
- A probabilidade de concorrência extrema é reduzida, tornando aceitável o custo associado ao bloqueio;
- A simplicidade e previsibilidade do comportamento transacional são adequadas ao contexto académico do projeto.

Implementação da Solução

A solução foi implementada através de uma transação explícita que:

- Bloqueia os registos da tabela `ProductVariant` no momento da leitura do stock;
- Garante que apenas uma transação pode validar e consumir variantes de um determinado produto de cada vez;
- Valida o cumprimento do `safety stock level` antes de permitir a venda;
- Impede leituras inconsistentes e atualizações concorrentes.

O nível de isolamento `SERIALIZABLE` assegura que:

- Não ocorrem leituras sujas (dirty reads);
- Não existem leituras não repetidas (non-repeatable reads);
- Não ocorrem leituras fantasma (phantom reads);

1ª Fase Relatório Técnico – Complementos de Bases de Dados

- A quantidade de variantes disponíveis permanece consistente durante toda a transação.

Demonstração

Para demonstrar o funcionamento da solução, foi efetuada uma simulação com múltiplas sessões concorrentes:

- Duas transações tentam, em simultâneo, reservar variantes do mesmo produto;
- A primeira transação bloqueia os registos relevantes de ProductVariant;
- A segunda transação fica em espera até a primeira concluir (COMMIT ou ROLLBACK);
- O sistema garante que o stock não é consumido abaixo do limite definido.

Demonstração do Conflito (Sem Controlo)

Sessão A – Venda sem Isolamento

Simular um utilizador que consulta o stock e efetua uma venda sem qualquer mecanismo de bloqueio.

O que acontece:

- O stock é lido sem bloqueio
- A transação fica suspensa durante 10 segundos
- Durante esse período, outras transações podem intervir
- A venda é registada sem validação concorrente

Sessão B – Venda Concorrente

Simular um segundo utilizador a vender o mesmo produto em simultâneo.

O que acontece

- Ambas as sessões leem o mesmo stock
- Ambas assumem disponibilidade
- As vendas são registadas em paralelo

Solução com Bloqueio Pessimista

Nível de isolamento: SERIALIZABLE

Tipo de controlo: Bloqueio pessimista

Objetivo: Garantir exclusividade durante a validação e registo da venda

Sessão A – Venda com Bloqueio

Objetivo

Bloquear o produto no momento da leitura e garantir acesso exclusivo até ao final da transação.

O que acontece

- O registo do produto é bloqueado
- Outras transações ficam impedidas de ler ou alterar o produto
- O stock é validado de forma segura
- A venda só é efetuada se respeitar o limite

Sessão B – Transação Concorrente Bloqueada

Objetivo

Demonstrar que uma transação concorrente é obrigada a aguardar.

O que acontece:

- A sessão fica bloqueada até a Sessão A terminar
- Apenas prossegue após o COMMIT
- O stock mantém-se consistente

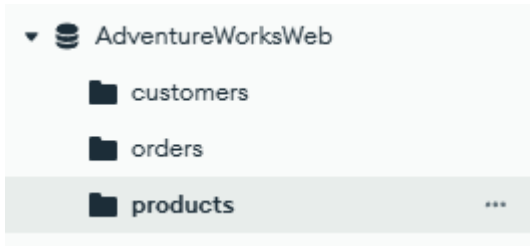
13. NoSQL

Collections usadas:

Customers - Criada para concentrar os dados base de identificação do cliente

Products - Criada para disponibilizar informação detalhada de produtos num formato adequado ao consumo web

Orders - representa o núcleo do “balanço de compras”, pois armazena as encomendas efetuadas



Queries de migração para JSON

Products:

SELECT

(

SELECT

PM.product_master_id AS _id,
PM.model AS model,
PM.description AS description,

PC.name AS category,
PSC.name AS subcategory,
PL.name AS product_line,
PCL.name AS class,

(

SELECT

PV.product_variant_id AS variant_id,
PV.variant_name,

1ª Fase Relatório Técnico – Complementos de Bases de Dados

COL.name AS color,
PS.name AS style,
PV.size,
SR.name AS size_range,
PV.size_unit_measure_code,
PV.weight,
PV.weight_unit_measure_code,
PV.standard_cost,
PV.list_price,
PV.dealer_price,
PV.days_to_manufacture,
PV.safety_stock_level,
PV.finished_goods_flag

FROM dbo.ProductVariant PV

LEFT JOIN dbo.ProductColor COL ON PV.color_id = COL.color_id

LEFT JOIN dbo.ProductStyle PS ON PV.style_id = PS.style_id

LEFT JOIN dbo.ProductSizeRange SR ON PV.size_range_id = SR.size_range_id

WHERE PV.product_master_id = PM.product_master_id

FOR JSON PATH

) AS variants

FROM dbo.ProductMaster PM

LEFT JOIN dbo.ProductCategory PC ON PM.category_id = PC.category_id

LEFT JOIN dbo.ProductSubcategory PSC ON PM.subcategory_id = PSC.subcategory_id

LEFT JOIN dbo.ProductLine PL ON PM.product_line_id = PL.product_line_id

LEFT JOIN dbo.ProductClass PCL ON PM.class_id = PCL.class_id

WHERE PM.model IS NOT NULL

FOR JSON PATH

) AS products_json;

Ano Letivo 2025/26

GO

Orders:

SELECT

(

SELECT TOP (100)

so.sales_order_id AS _id,

so.sales_order_number,

so.customer_id,

so.sales_territory_id,

so.order_date,

so.due_date,

so.ship_date,

(

SELECT

sol.sales_order_line_id AS _id,

sol.line_number,

sol.product_variant_id,

sol.product_standard_cost,

sol.unit_price,

sol.quantity,

sol.tax_amt,

sol.freight

FROM dbo.SalesOrderLine AS sol

WHERE sol.sales_order_id = so.sales_order_id

FOR JSON PATH

) AS lines

1ª Fase Relatório Técnico – Complementos de Bases de Dados

```
FROM dbo.SalesOrder AS so
ORDER BY so.sales_order_id
FOR JSON PATH, ROOT('orders')
) AS orders_json;
GO
```

Customers:

```
SELECT
(
    SELECT TOP 2
        customer_id AS _id,
        email_address,
        first_name,
        last_name,
        birth_date,
        number_cars_owned
    FROM dbo.Customer
    WHERE email_address IS NOT NULL
    FOR JSON AUTO, ROOT('customers')) AS test;
GO
```

Queries Usadas:

Resultado excluindo explicitamente valores omissos e ordenado

Excluir docs onde category é null/vazio e ordenar por model

Find { category: { \$exists: true, \$nin: [null, ""] } }

Project { _id: 1, model: 1, category: 1, subcategory: 1, product_line: 1, class: 1 }

Sort { model: 1 }



```
{ category: { $exists: true, $nin: [null, ""] } }
```

[Generate query](#) 

Explain

Reset

Find



[Options](#) 

Project

```
{ _id: 1, model: 1, category: 1, subcategory: 1, product_line: 1, class: 1 }
```

Sort

```
{ model: 1 }
```

Max Time MS

Collation

```
{ locale: 'simple' }
```

Skip

Limit

Index Hint

```
{ field: -1 }
```

 EXPORT DATA 

25 

 1 – 25 of 119       

```
_id: 1
model: "All-Purpose Bike Stand"
category: "Accessories"
subcategory: "Bike Stands"
product_line: "M"
```

```
_id: 2
model: "Bike Wash"
category: "Accessories"
subcategory: "Cleaners"
product_line: "S"
```

```
_id: 3
model: "Cable Lock"
category: "Accessories"
subcategory: "Locks"
product_line: "S"
```

1ª Fase Relatório Técnico – Complementos de Bases de Dados

Envolve uma filtragem

Produtos de uma categoria específica (ex. "Bikes") e com variantes

Find { category: "Bikes", variants: { \$exists: true, \$ne: [] } }

```
Project { _id: 1, model: 1, category: 1, "variants.variant_id": 1, "variants.list_price": 1 }
```

Sort { model: 1 }

Query Builder

Filter: { category: "Bikes", variants: { \$exists: true, \$ne: [] } } [Generate query](#) [Explain](#) [Reset](#) [Find](#) [Options](#)

Project: { _id: 1, model: 1, category: 1, "variants.variant_id": 1, "variants.list_price": 1 }

Sort: { model: 1 } **Max Time MS:** 60000

Collation: { locale: 'simple' } **Skip:** 0 **Limit:** 0

Index Hint: { field: -1 }

[EXPORT DATA](#) 25 1 – 15 of 15 [Refresh](#) [Previous](#) [Next](#) [Menu](#) [JSON](#) [Table](#)

```

_id: 87
model : "Mountain-100"
category : "Bikes"
▶ variants : Array (8)
    
```

```

_id: 88
model : "Mountain-200"
category : "Bikes"
▶ variants : Array (12)
    
```

```

_id: 89
model : "Mountain-300"
category : "Bikes"
▶ variants : Array (4)
    
```

```

_id: 90
model : "Mountain-400-W"
    
```

Envolve uma Agregação

Total de variantes por categoria (e ordena desc)

```
[
  { $match: { category: { $exists: true, $nin: [null, ""] } } },
  { $unwind: { path: "$variants", preserveNullAndEmptyArrays: false } },
  { $group: { _id: "$category", total_variants: { $sum: 1 } } },
  { $sort: { total_variants: -1 } }
]
```

📁

\$match

\$unwind

\$group

\$sort

Edit

Explain

Export

Run

Options ▶

ALL RESULTS

Showing 1 – 4 count results < > ▾ ≡ { }

<pre>_id: "Components" total_variants : 189</pre>
<pre>_id: "Bikes" total_variants : 125</pre>
<pre>_id: "Clothing" total_variants : 48</pre>
<pre>_id: "Accessories" total_variants : 35</pre>

14. Descrição da Demonstração

14.1.Script de demonstração

Sequencia de execução do Código

1. Criação da Base de Dados e Estrutura (Script de Estrutura)

Correr o ficheiro create_db.sql dentro da pasta CBD-Projeto\sql_scripts\schema

2. Execução das funções de ajuda á migração

Correr o ficheiro functions.sql dentro da pasta CBD-Projeto\sql_scripts\schema

3. Migração dos dados da base de dados antiga para a nova

Dentro da pasta CBD-Projeto\sql_scripts\migration correr sequencialmente o ficheiro migrate_products, migrate_customers, migrate_sales e migrate_app_users.sql

Depois correr o ficheiro statistics.sql que está dentro da pasta CBD-Projeto\sql_scripts\schema

4. Validações

Correr todos os ficheiros sql que estão dentro da pasta CBD-Projeto\sql_scripts\validations

15. Conclusões

A realização deste projeto permitiu desenvolver um processo completo de migração e reconstrução de uma base de dados relacional, garantindo a integridade, consistência e segurança dos dados. Através da análise e reformulação do modelo de dados da base AdventureWorksLegacy, foi possível criar uma nova estrutura normalizada (AdventureWorks), com todas as relações e restrições adequadas a um ambiente de produção.

Durante a Fase 1, foram implementadas funções, stored procedures e triggers em T-SQL para apoiar o processo de migração e assegurar a qualidade dos dados importados. Destacam-se mecanismos de validação, controlo de duplicados, normalização de valores e proteção de dados sensíveis, garantindo que a informação migrada manteve coerência e fiabilidade.

A Fase 2 permitiu aprofundar aspetos fundamentais de otimização e administração da base de dados. Foram definidos e implementados índices adequados às principais queries analíticas, com base na análise de seletividade e densidade dos dados, permitindo melhorar significativamente o desempenho das consultas relacionadas com vendas, produtos e clientes. A comparação dos planos de execução antes e depois da criação dos índices evidenciou a redução de custos de execução e de acessos desnecessários às tabelas.

Adicionalmente, foi definida uma estratégia de backup e recuperação, contemplando backups completos, diferenciais e de log, bem como a simulação de um cenário de falha e respetiva recuperação da base de dados. Foram também implementados níveis diferenciados de acesso à informação, através da criação de utilizadores, roles e permissões específicas, garantindo o princípio do menor privilégio para perfis como Administrador, SalesPerson e SalesTerritory.

No âmbito da segurança, foram aplicadas técnicas de encriptação e hashing para campos sensíveis, como passwords, NIF e perguntas de recuperação de conta, assegurando a confidencialidade dos dados pessoais. Foram ainda definidos mecanismos de controlo de transações e níveis de isolamento adequados a cenários de concorrência.

Por fim, a integração com um sistema NoSQL (MongoDB) permitiu demonstrar a complementaridade entre bases de dados relacionais e não relacionais, possibilitando a disponibilização de informação para fins de reporting e visualização sem sobrecarregar o sistema transacional.

Em suma, o projeto atingiu plenamente os seus objetivos, constituindo uma experiência prática abrangente na aplicação de conceitos de engenharia de dados, otimização de desempenho, segurança, administração de bases de dados e integração de tecnologias, evidenciando a importância de boas práticas no desenho, manutenção e evolução de sistemas de informação empresariais.

